

	CH20	SCC	FAF	TUE	CALC	MTRANS	Check
0	549	0	0	840	3	3	1
1	1267	1	1189	0	2	3	1
2	549	0	1071	840	1	3	1
3	549	0	1071	0	1	4	0
4	549	0	0	0	2	3	0

```
df.tail()
```

	Gender	Age	Height	Weight	family_history	FAVC	FCVC	NCP
CAEC	\							
2106	0	397	779	1412	1	1	809	477
2								
2107	0	572	983	1453	1	1	809	477
2								
2108	0	617	1006	1451	1	1	809	477
2								
2109	0	812	923	1437	1	1	809	477
2								
2110	0	754	919	1441	1	1	809	477
2								

	SMOKE	CH20	SCC	FAF	TUE	CALC	MTRANS	Check
2106	0	387	0	924	747	2	3	0
2107	0	560	0	761	467	2	3	0
2108	0	615	0	788	503	2	3	0
2109	0	1181	0	661	458	2	3	0
2110	0	1189	0	601	568	2	3	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                2111 non-null  int32
1   Age                  2111 non-null  int64
2   Height               2111 non-null  int64
3   Weight              2111 non-null  int64
4   family_history       2111 non-null  int32
5   FAVC                2111 non-null  int32
6   FCVC                2111 non-null  int64
7   NCP                 2111 non-null  int64
8   CAEC                2111 non-null  int32
9   SMOKE               2111 non-null  int32
10  CH20                2111 non-null  int64
11  SCC                 2111 non-null  int32
12  FAF                 2111 non-null  int64
```

```

13  TUE                2111 non-null    int64
14  CALC               2111 non-null    int32
15  MTRANS             2111 non-null    int32
16  Check              2111 non-null    int64

```

```
dtypes: int32(8), int64(9)
```

```
memory usage: 214.5 KB
```

```
df.describe()
```

	Gender	Age	Height	Weight
family_history \				
count	2111.000000	2111.000000	2111.000000	2111.000000
mean	0.505921	646.025107	753.162956	641.452866
std	0.500083	407.870050	453.399155	437.449707
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	304.500000	364.000000	264.500000
50%	1.000000	650.000000	711.000000	556.000000
75%	1.000000	982.000000	1144.500000	1004.500000
max	1.000000	1401.000000	1573.000000	1524.000000

	FAVC	FCVC	NCP	CAEC	SMOKE
\					
count	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000
mean	0.883941	457.986736	388.715775	1.859308	0.020843
std	0.320371	295.226514	179.959492	0.468543	0.142893
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	170.000000	325.500000	2.000000	0.000000
50%	1.000000	421.000000	477.000000	2.000000	0.000000
75%	1.000000	809.000000	477.000000	2.000000	0.000000
max	1.000000	809.000000	634.000000	3.000000	1.000000

	CH20	SCC	FAF	TUE	CALC
\					
count	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000

mean	601.051161	0.045476	539.049266	482.125533	2.268593
std	391.006783	0.208395	407.078823	402.671537	0.515498
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	312.500000	0.000000	112.500000	0.000000	2.000000
50%	549.000000	0.000000	589.000000	487.000000	2.000000
75%	905.500000	0.000000	919.500000	840.000000	3.000000
max	1267.000000	1.000000	1189.000000	1128.000000	3.000000

	MTRANS	Check
count	2111.000000	2111.000000
mean	2.365230	0.264803
std	1.261423	0.441333
min	0.000000	0.000000
25%	3.000000	0.000000
50%	3.000000	0.000000
75%	3.000000	1.000000
max	4.000000	1.000000

df.drop_duplicates()

	Gender	Age	Height	Weight	family_history	FAVC	FCVC	NCP
CAEC \								
0	0	404	295	245	1	0	170	477
2								
1	0	404	28	173	1	0	809	477
2								
2	1	701	1306	382	1	0	170	477
2								
3	1	1021	1306	643	0	0	809	477
2								
4	1	578	1202	692	0	0	170	0
2								
...	...	...	...	...	...	...	...	...
...								
2106	0	397	779	1412	1	1	809	477
2								
2107	0	572	983	1453	1	1	809	477
2								
2108	0	617	1006	1451	1	1	809	477
2								
2109	0	812	923	1437	1	1	809	477
2								
2110	0	754	919	1441	1	1	809	477

	SMOKE	CH20	SCC	FAF	TUE	CALC	MTRANS	Check
0	0	549	0	0	840	3	3	1
1	1	1267	1	1189	0	2	3	1
2	0	549	0	1071	840	1	3	1
3	0	549	0	1071	0	1	4	0
4	0	549	0	0	0	2	3	0
...	...	...	...	...	...	...	...	...
2106	0	387	0	924	747	2	3	0
2107	0	560	0	761	467	2	3	0
2108	0	615	0	788	503	2	3	0
2109	0	1181	0	661	458	2	3	0
2110	0	1189	0	601	568	2	3	0

```
df.isnull()
```

[illegible][illegible]

2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
2106	False	False	False	False	False	False	False	False	False
2107	False	False	False	False	False	False	False	False	False
2108	False	False	False	False	False	False	False	False	False
2109	False	False	False	False	False	False	False	False	False
2110	False	False	False	False	False	False	False	False	False

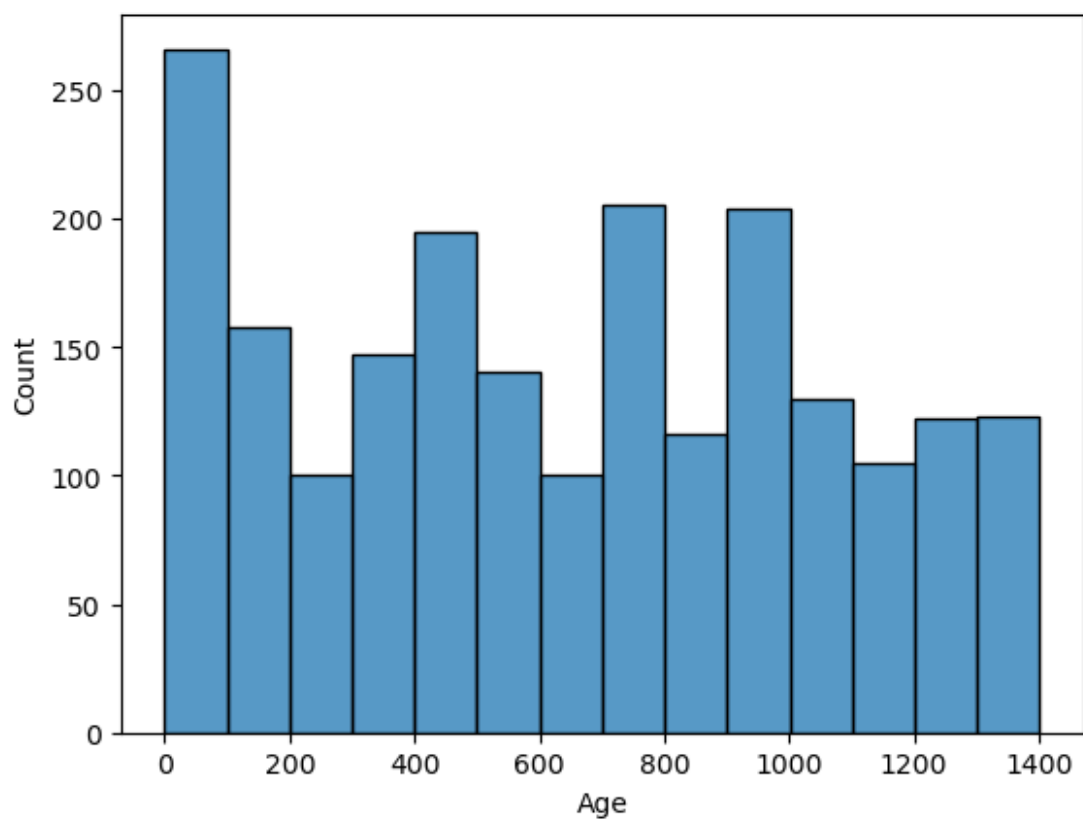
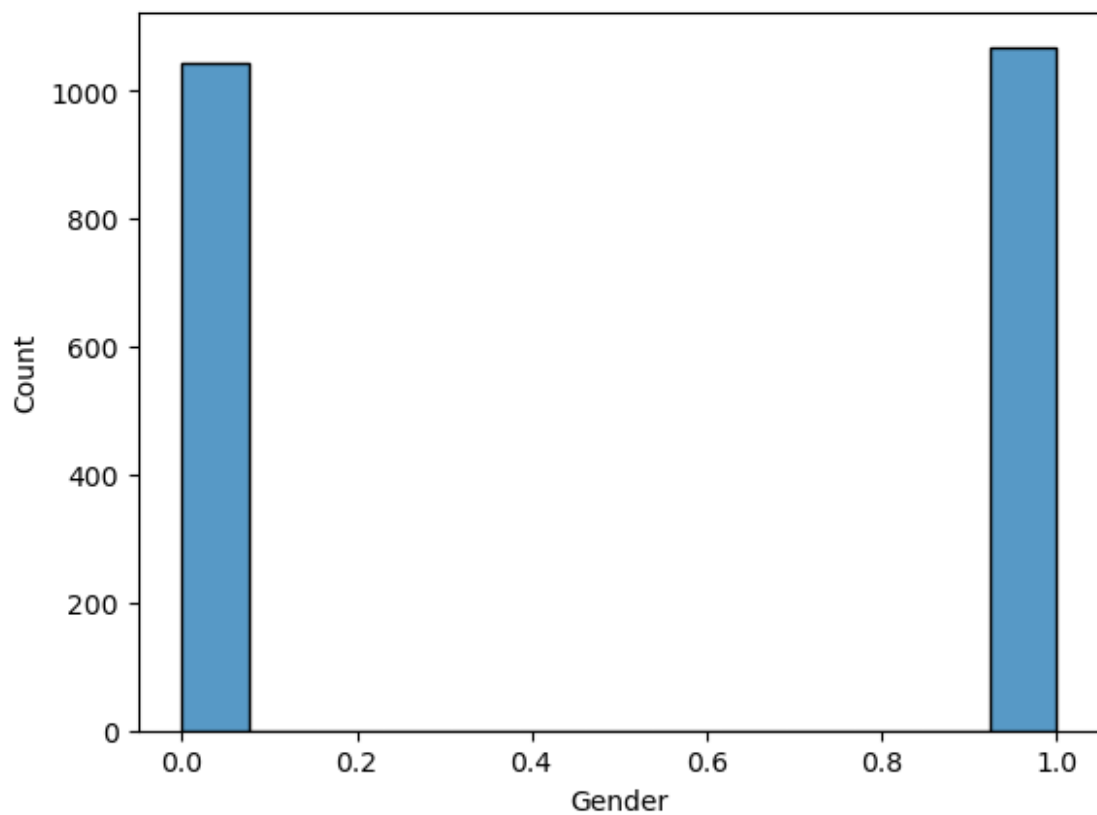
[2111 rows x 17 columns]

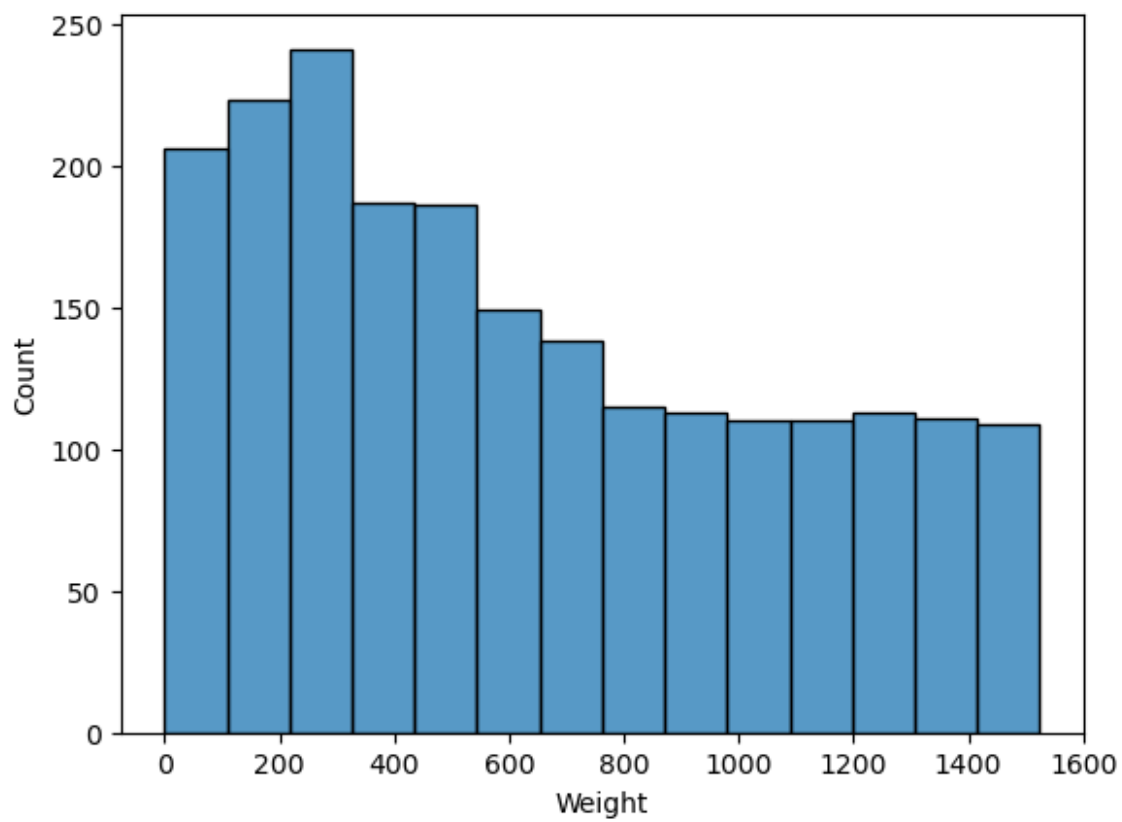
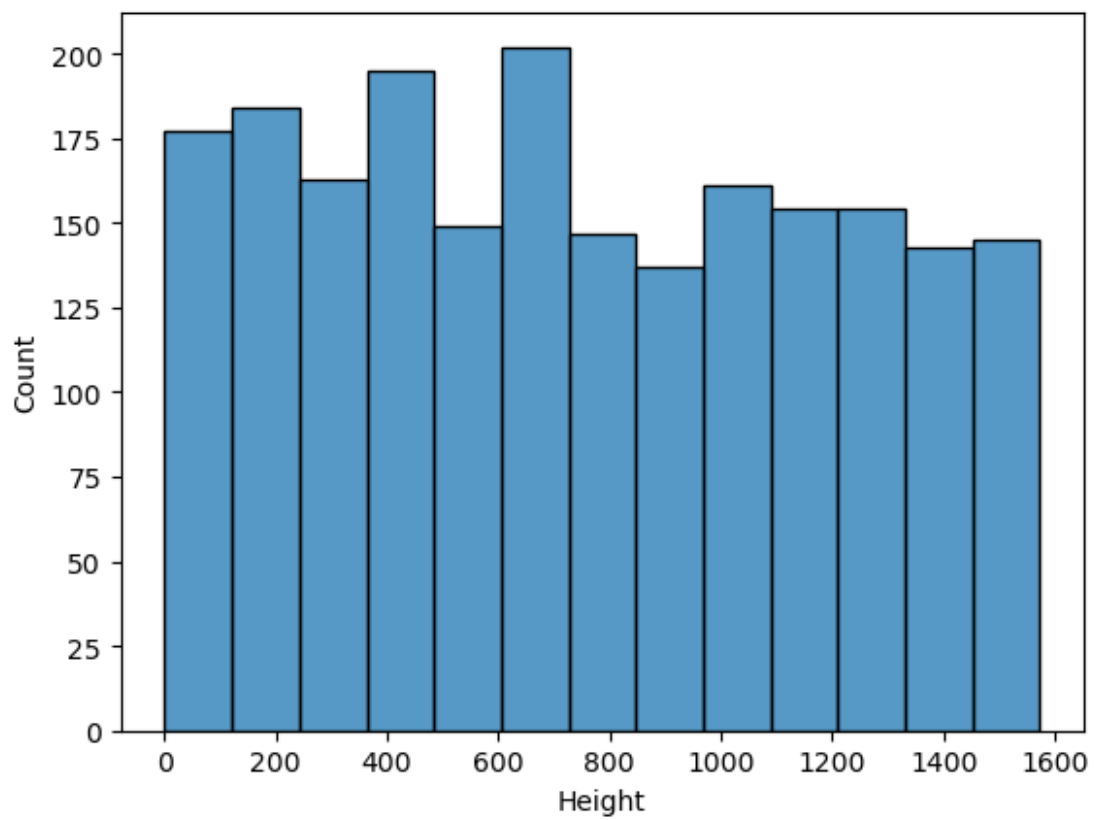
```
df.isnull().sum()
```

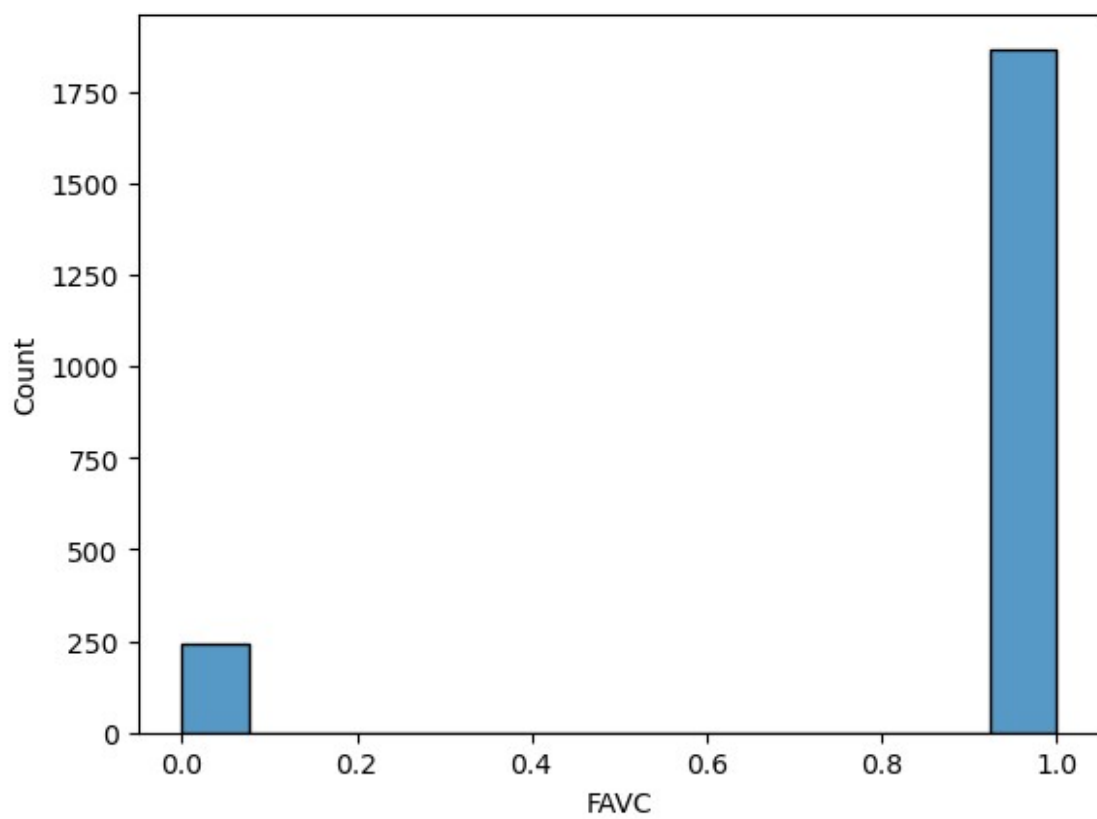
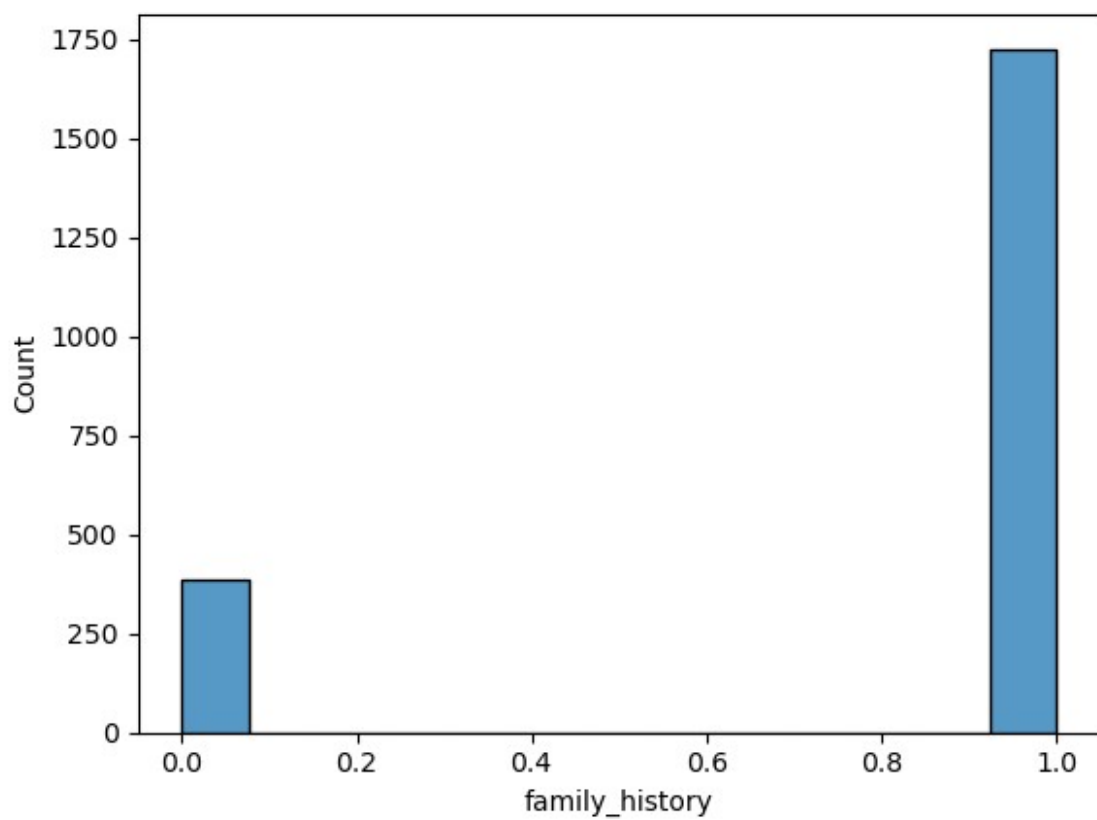
Gender	0
Age	0
Height	0
Weight	0
family_history	0
FAVC	0
FCVC	0
NCP	0
CAEC	0
SMOKE	0
CH20	0
SCC	0
FAF	0
TUE	0
CALC	0
MTRANS	0
Check	0

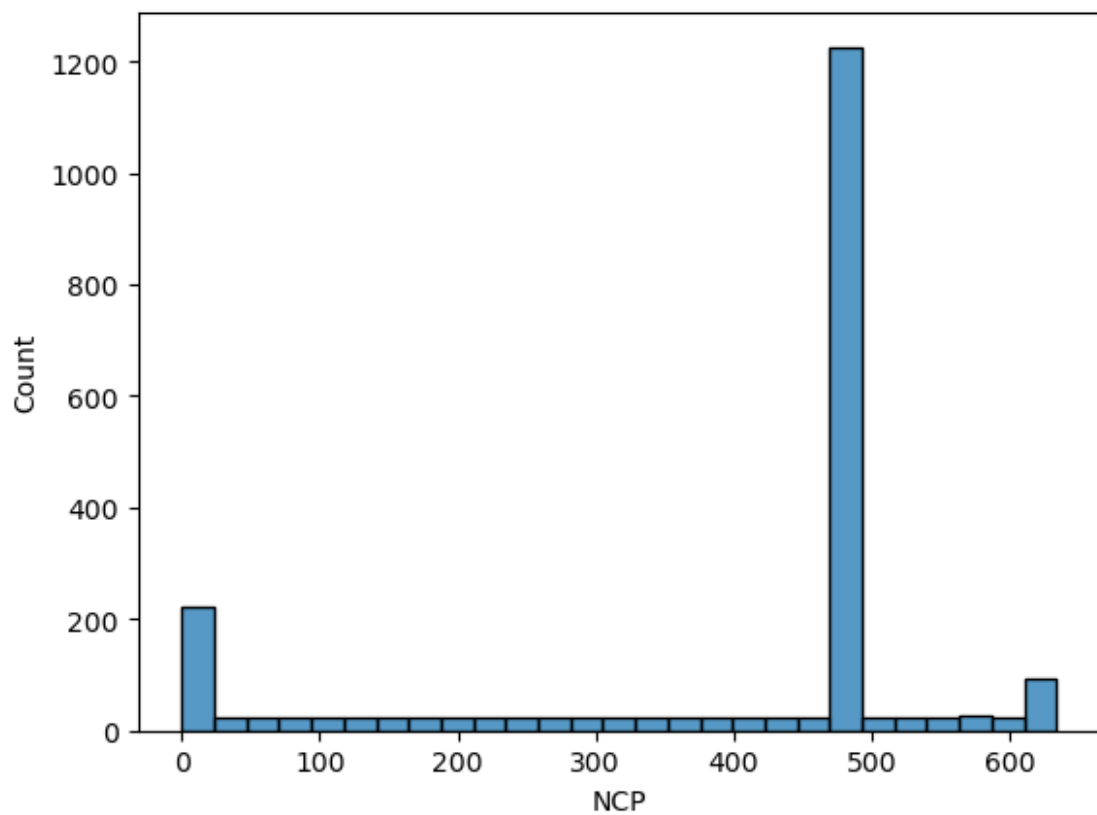
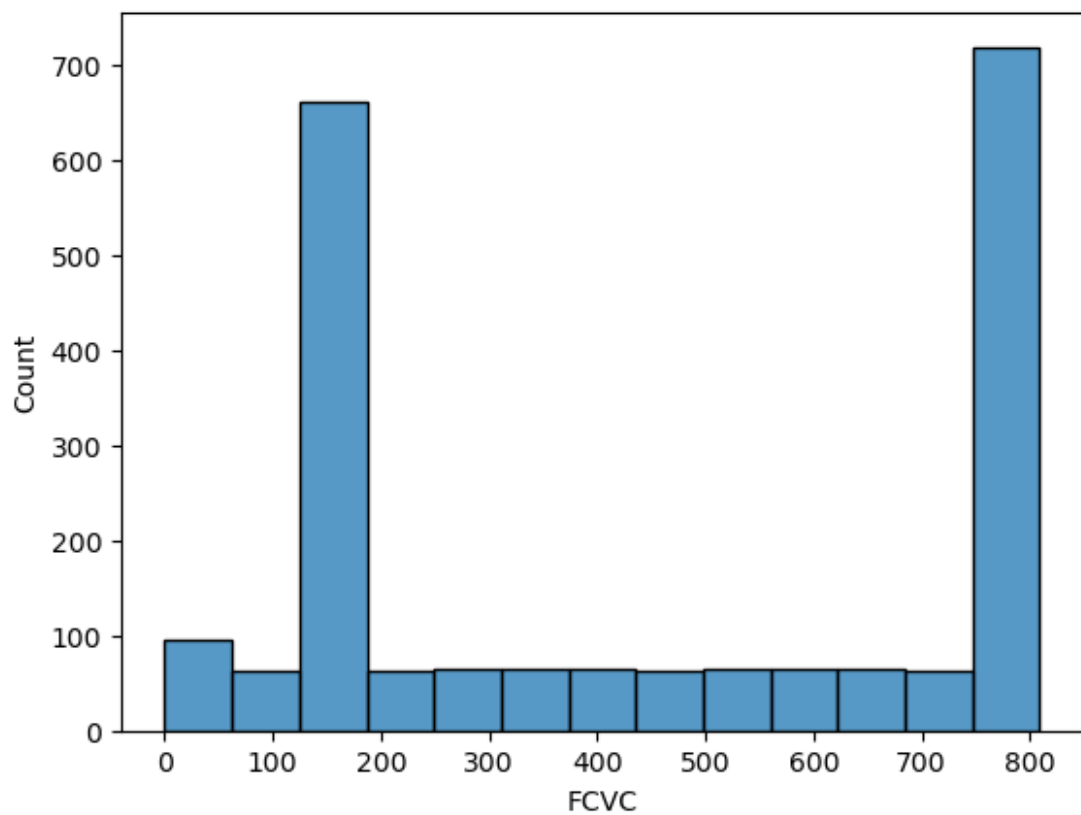
dtype: int64

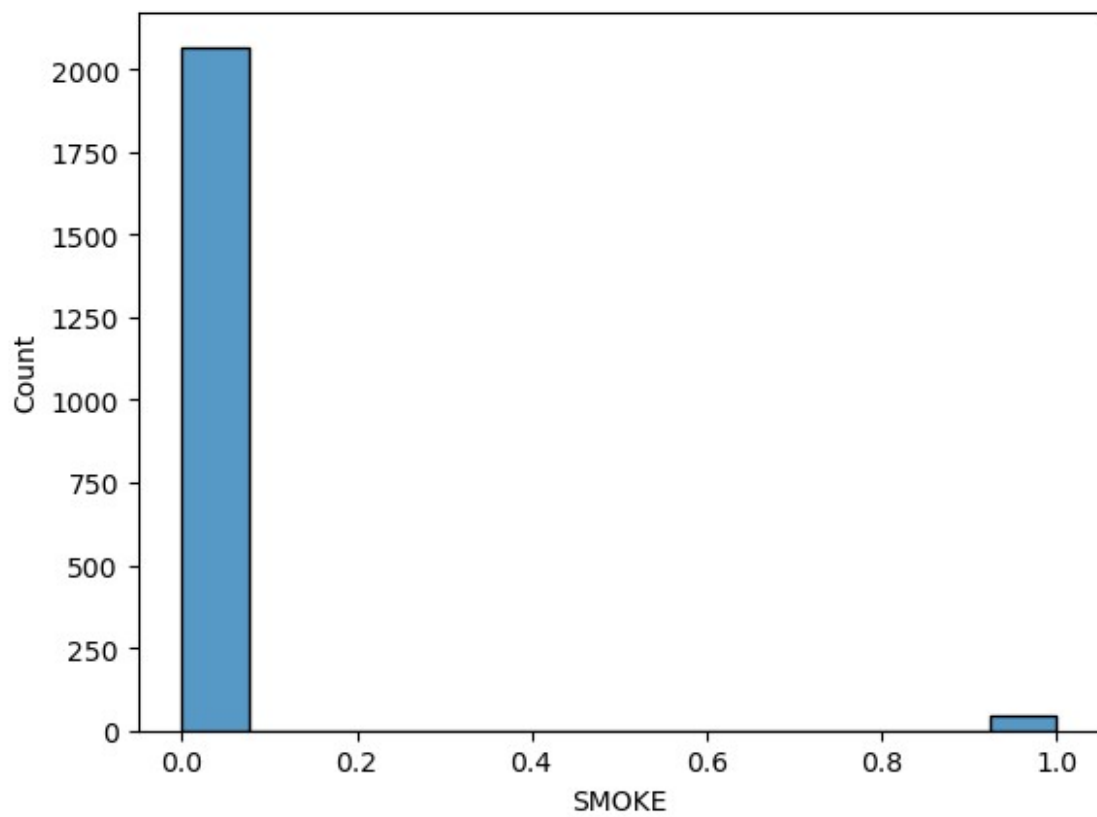
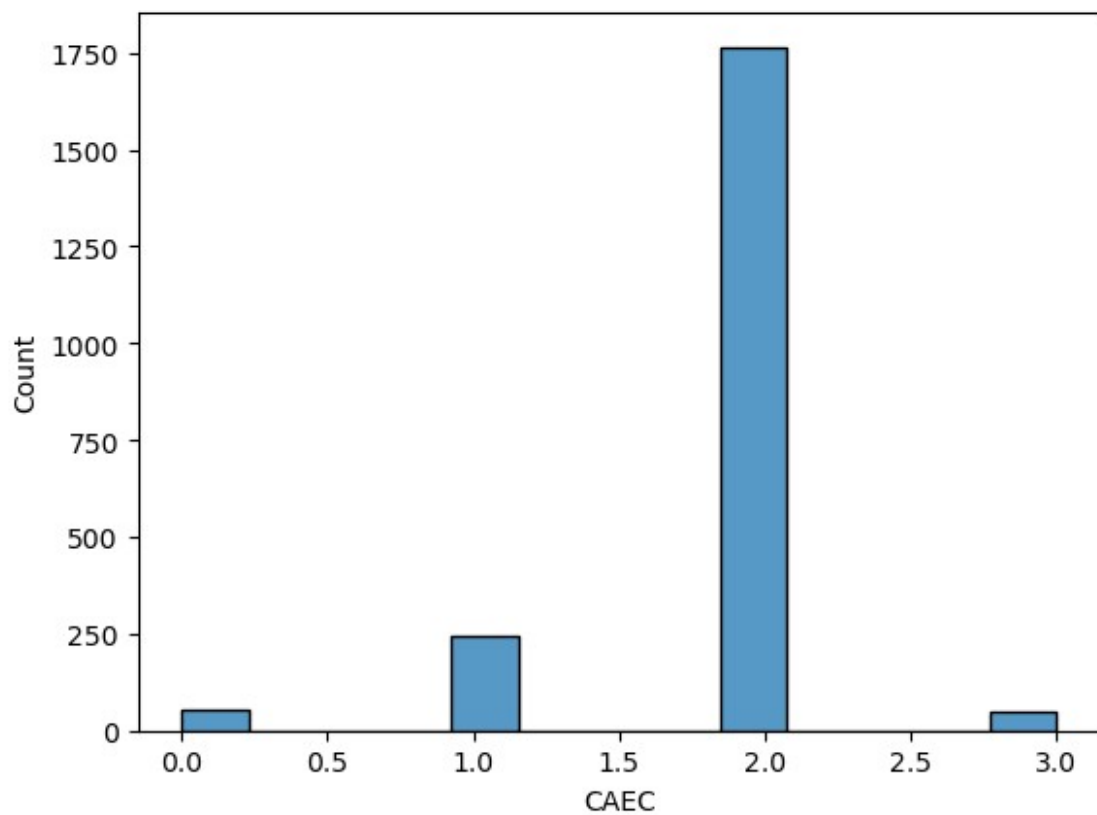
```
for i in df.select_dtypes(include="int").columns:
    sns.histplot(data=df, x=i)
    plt.show()
```

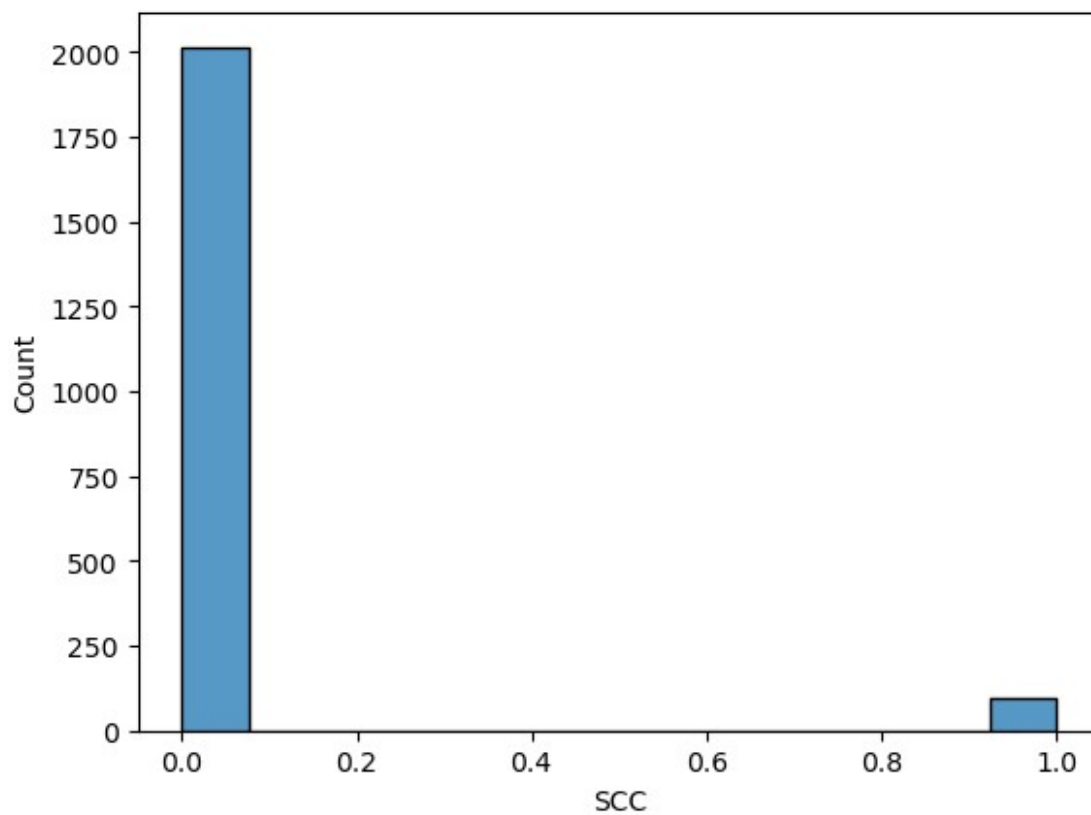
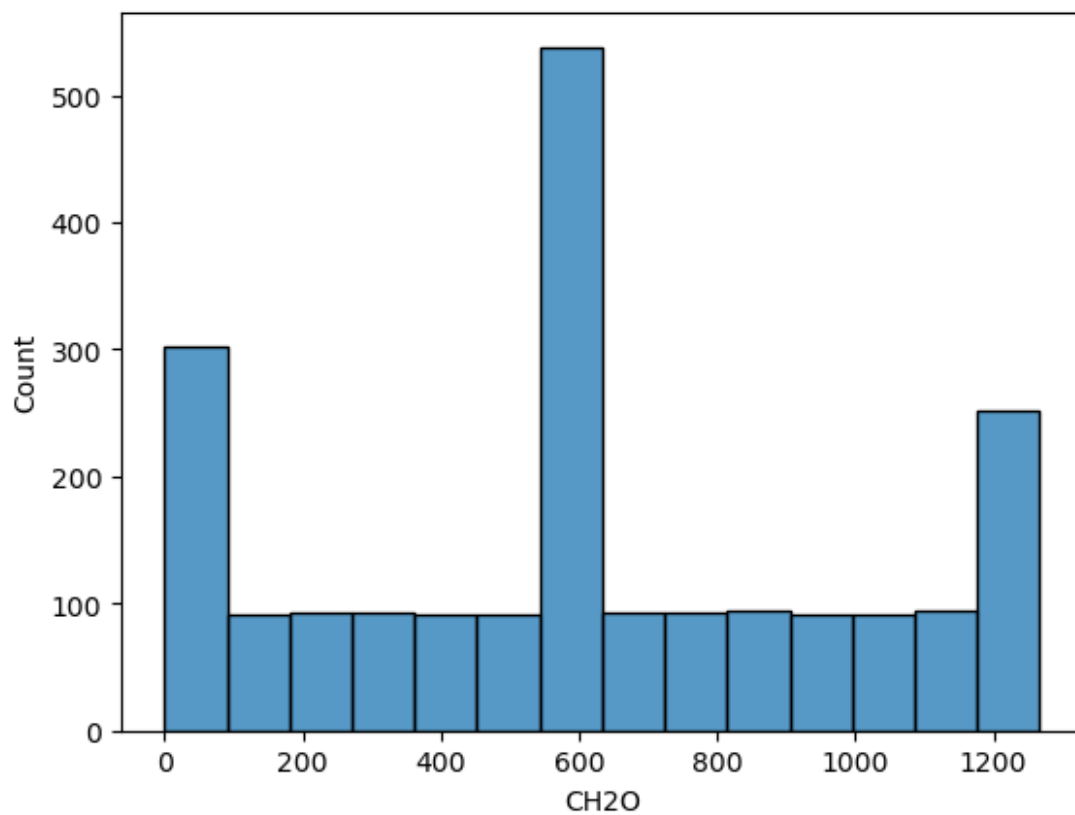


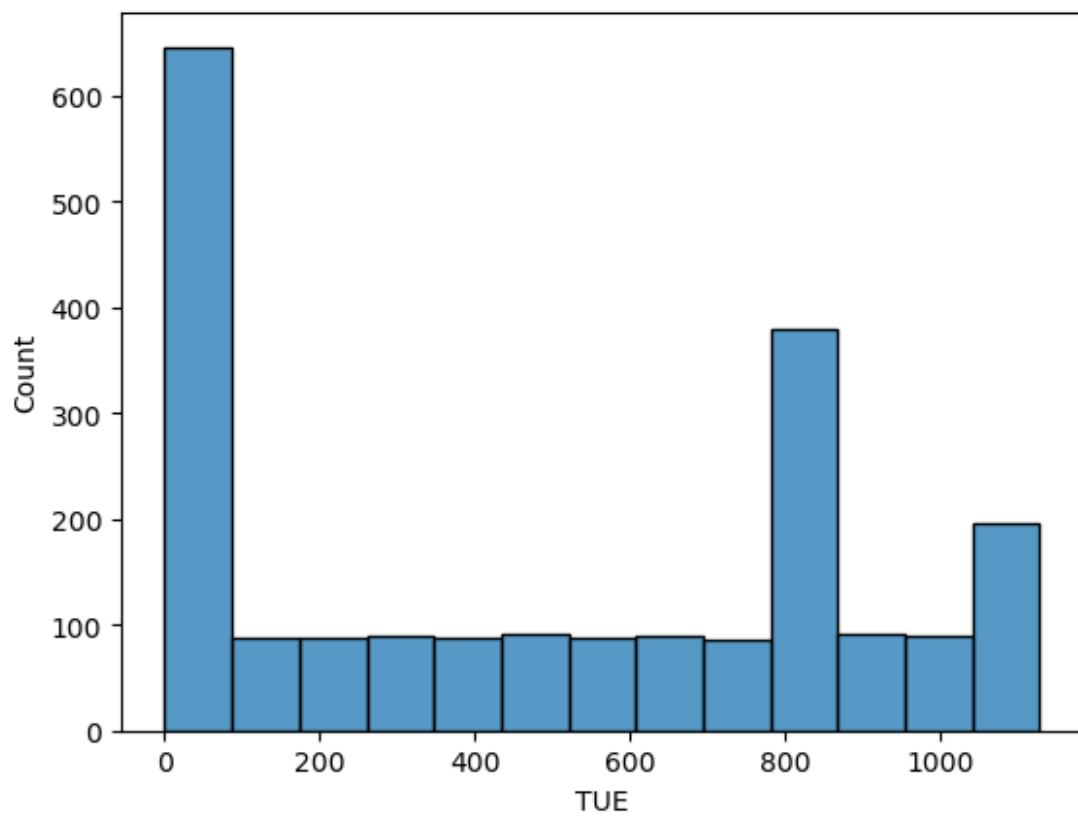
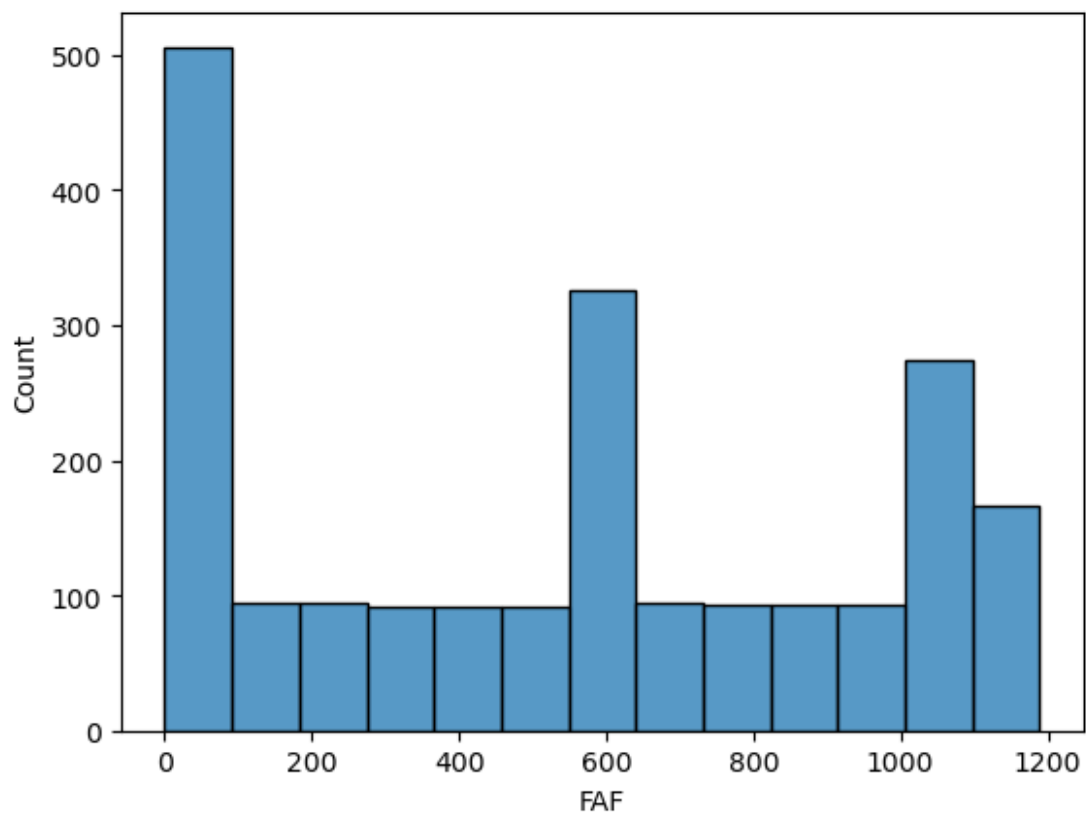


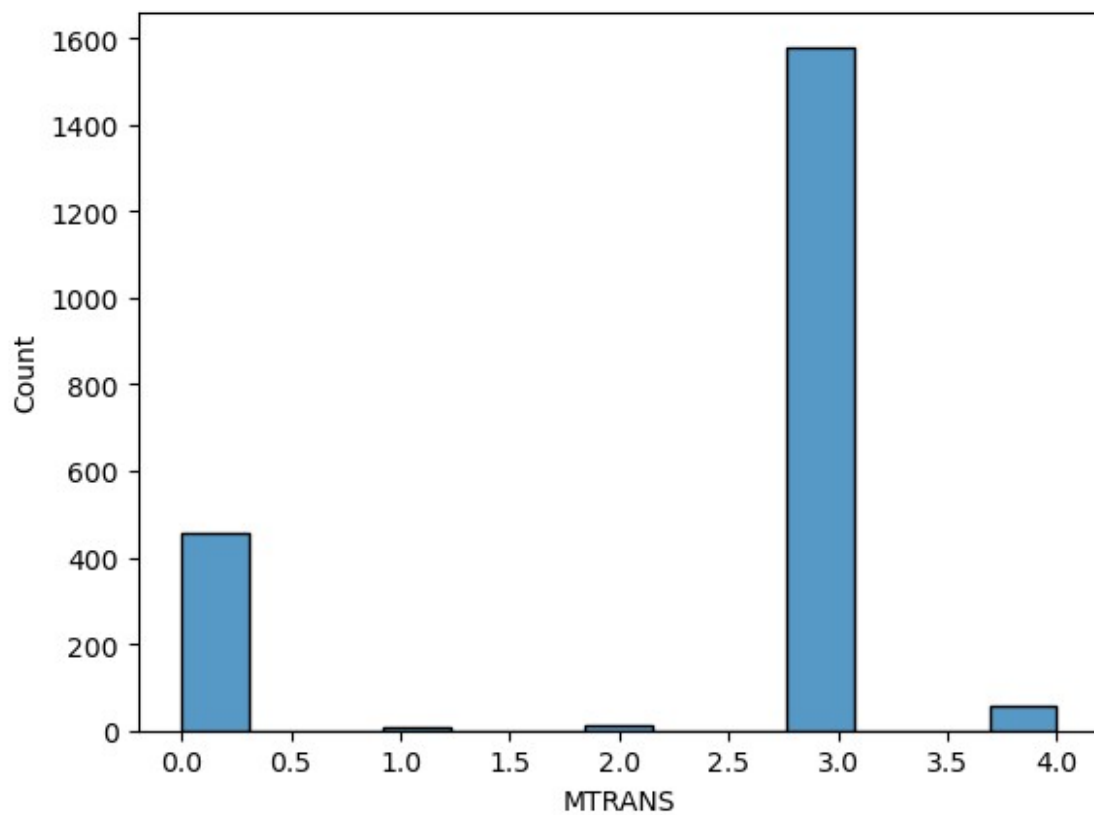
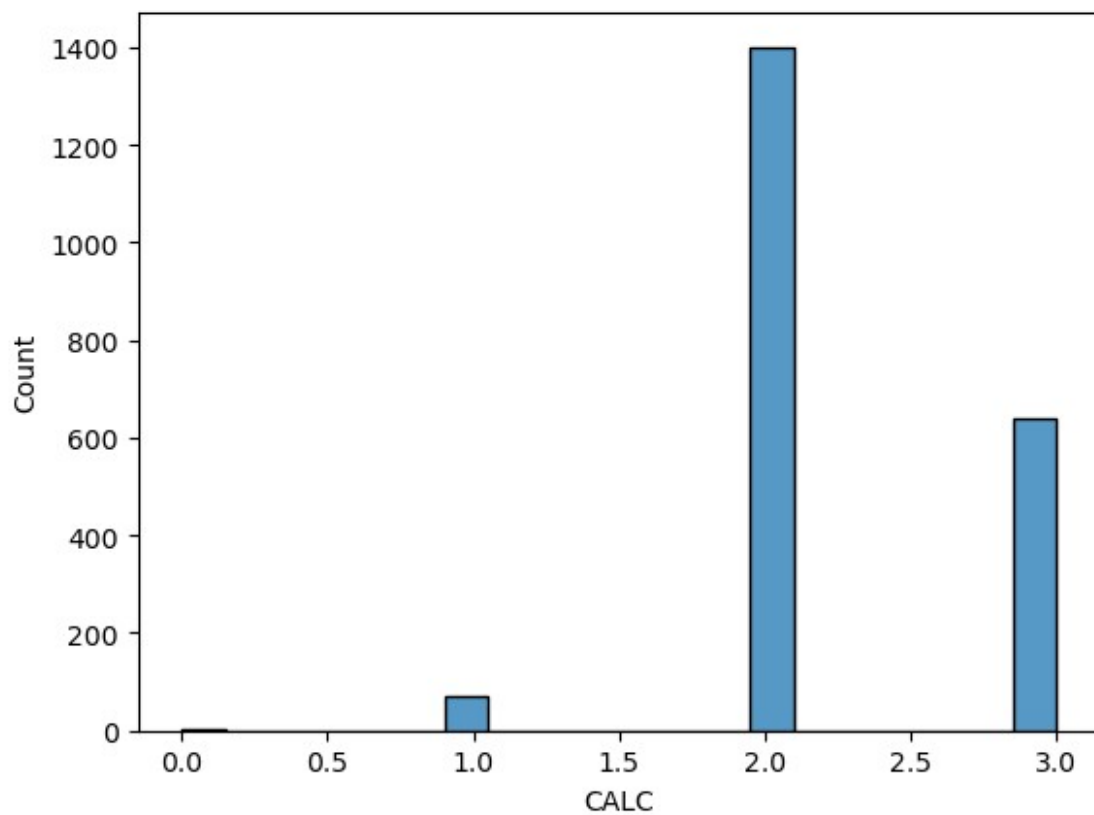


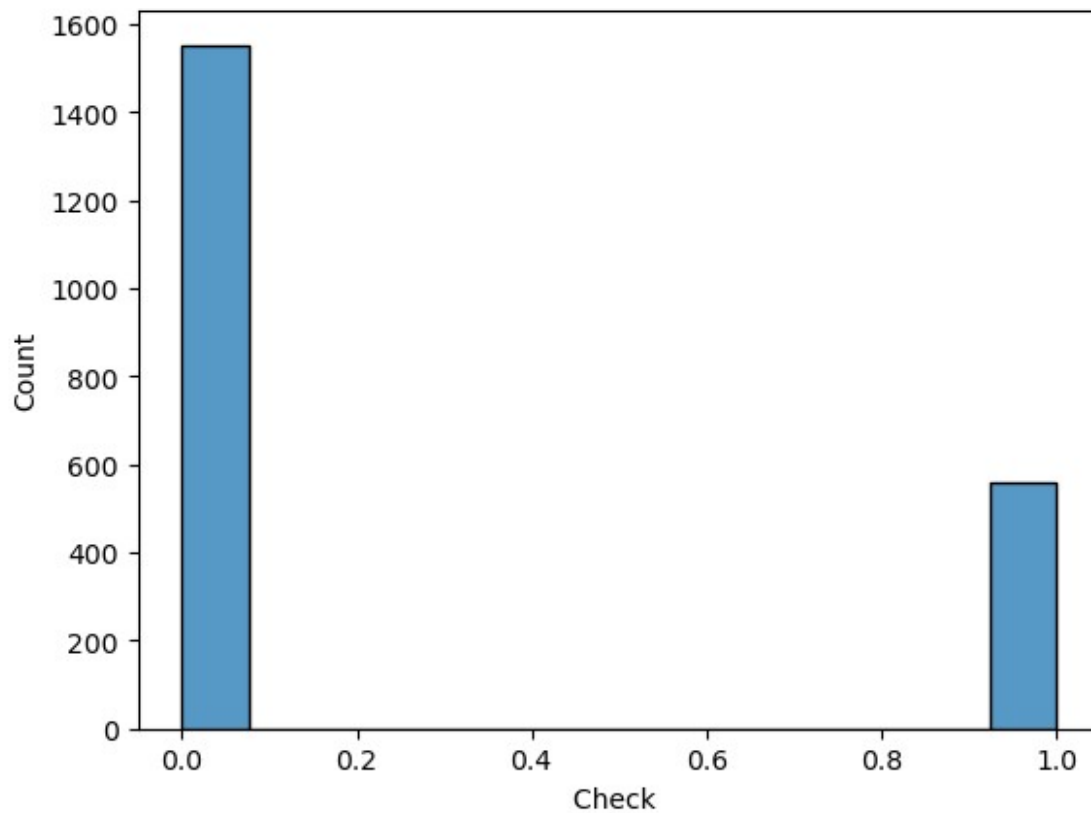




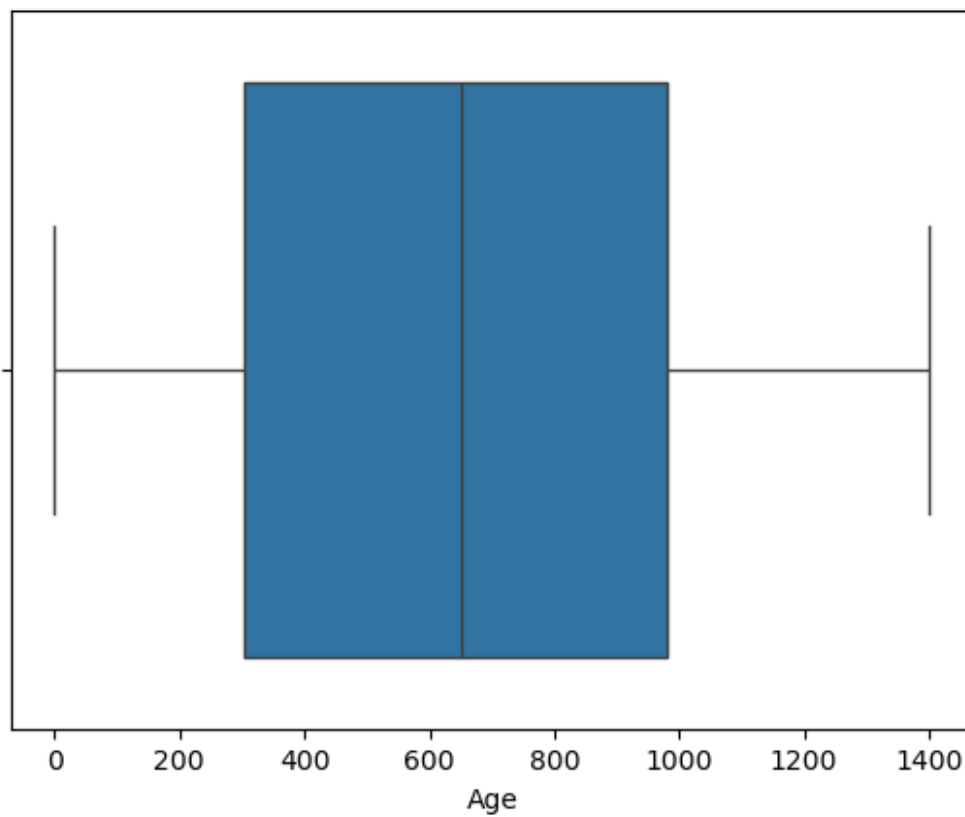
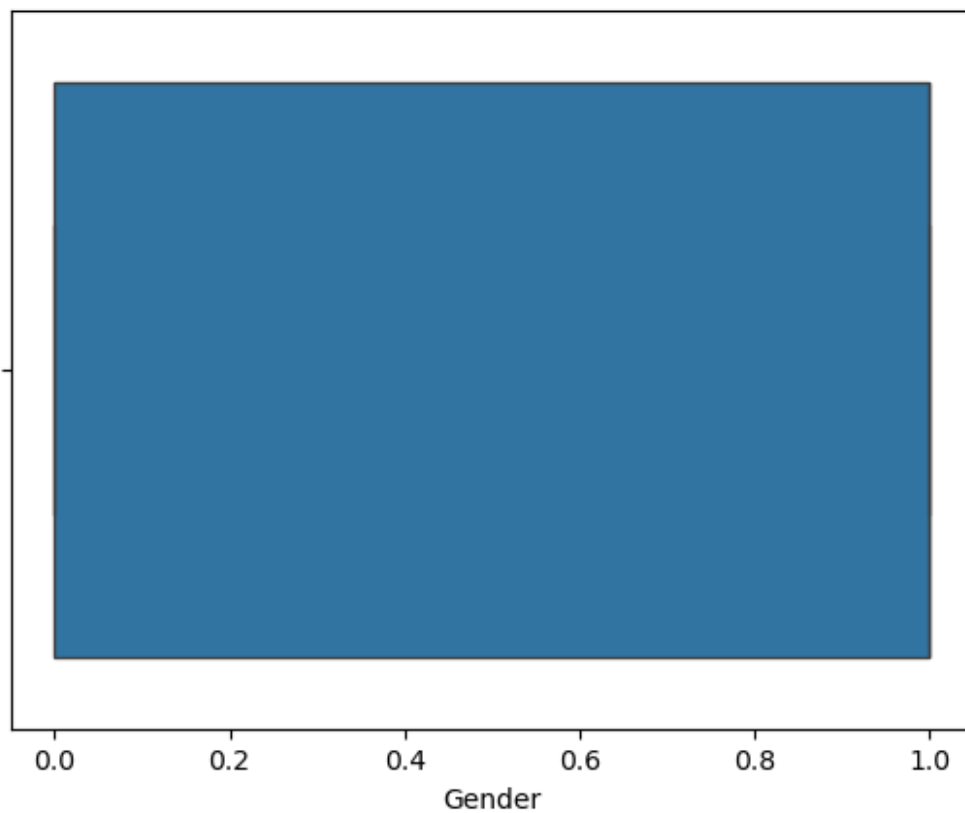


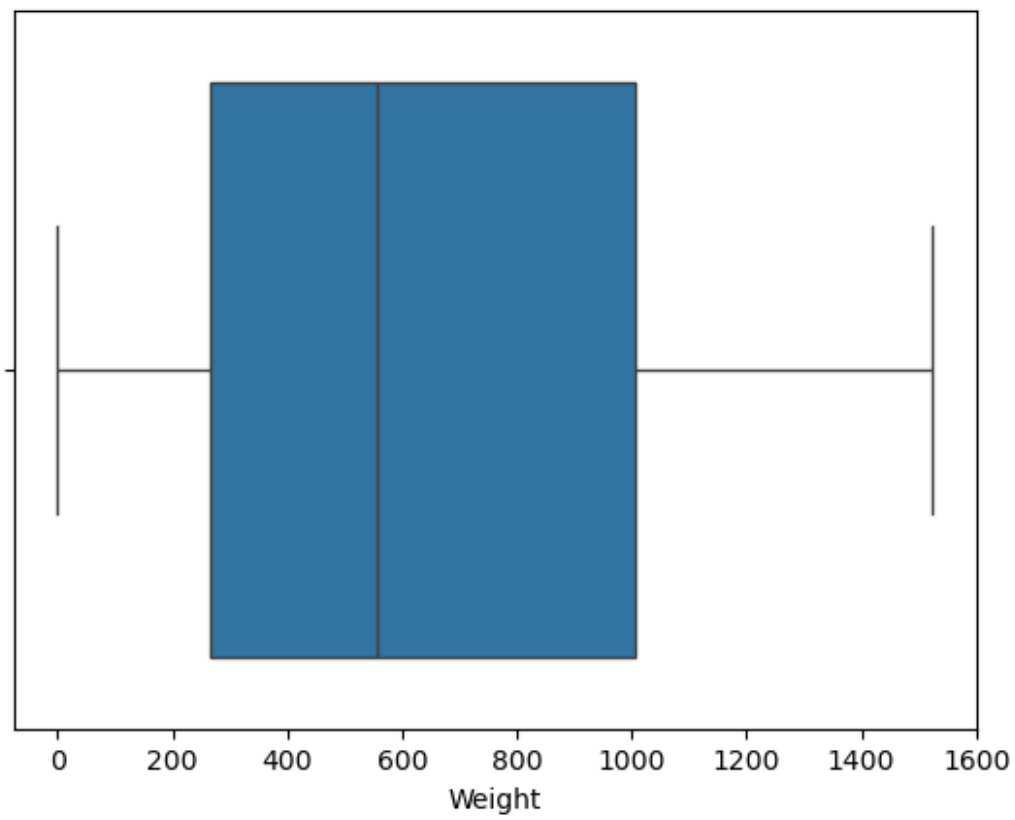
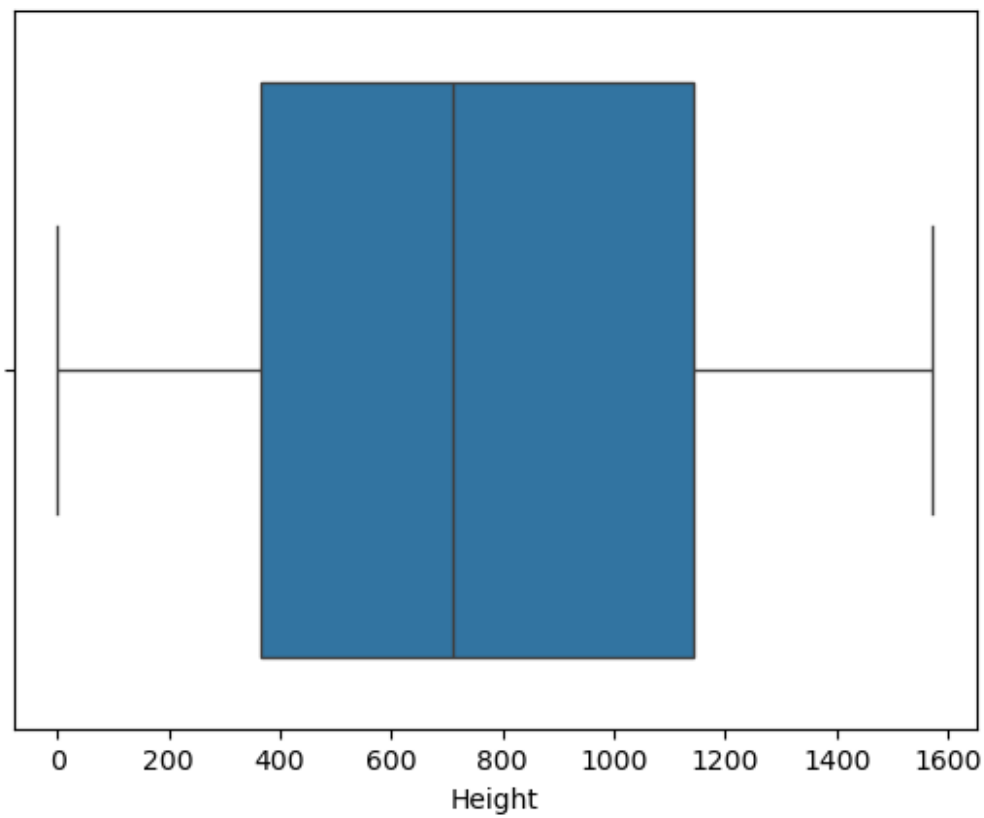


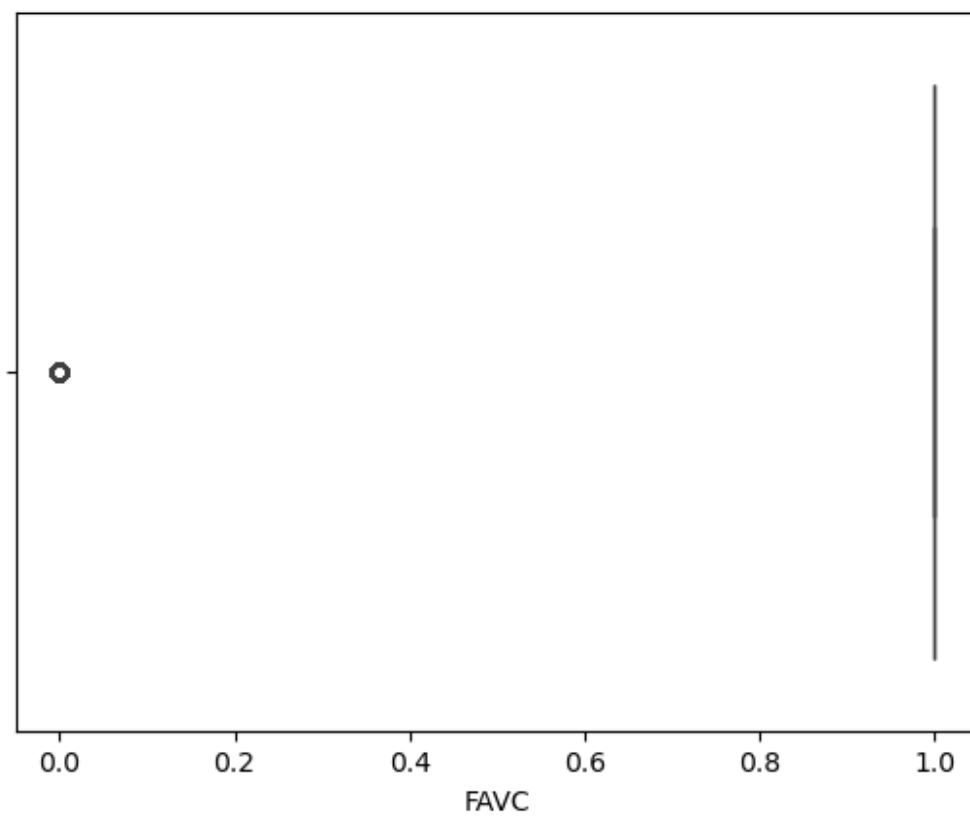
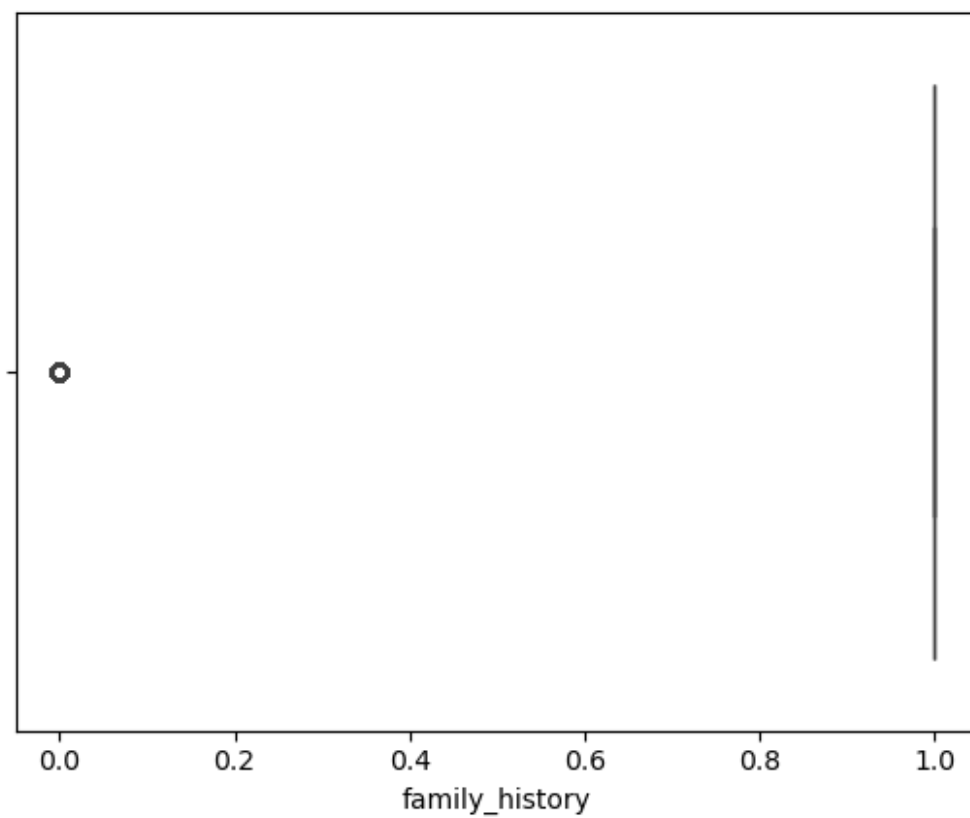


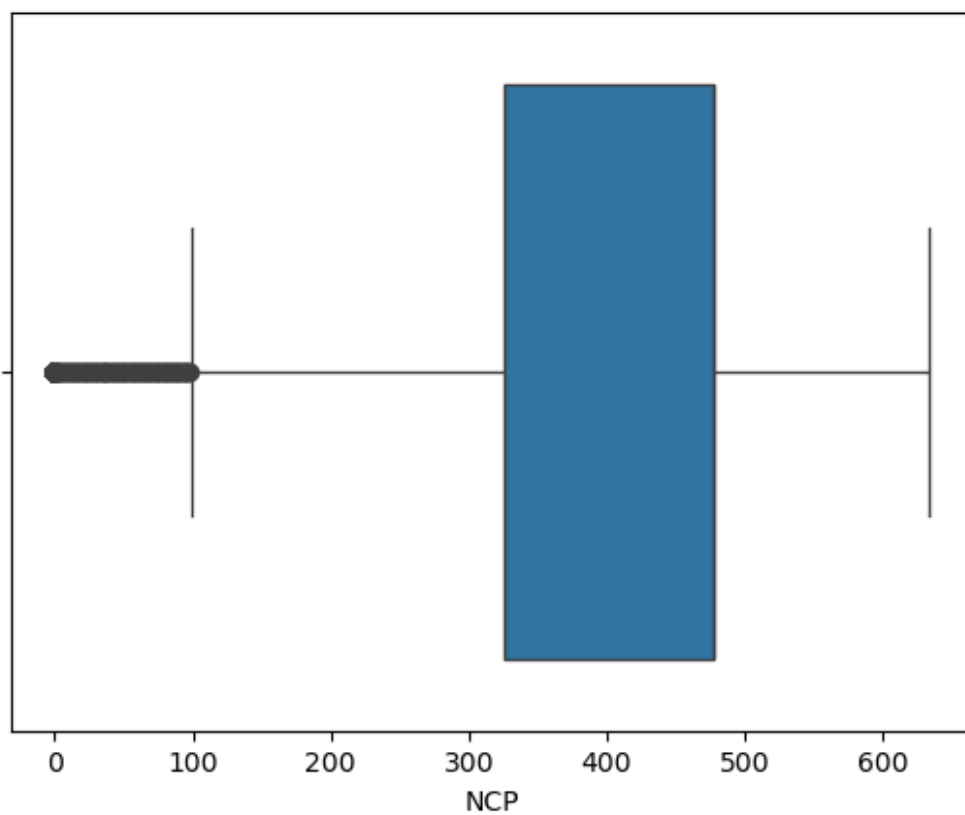
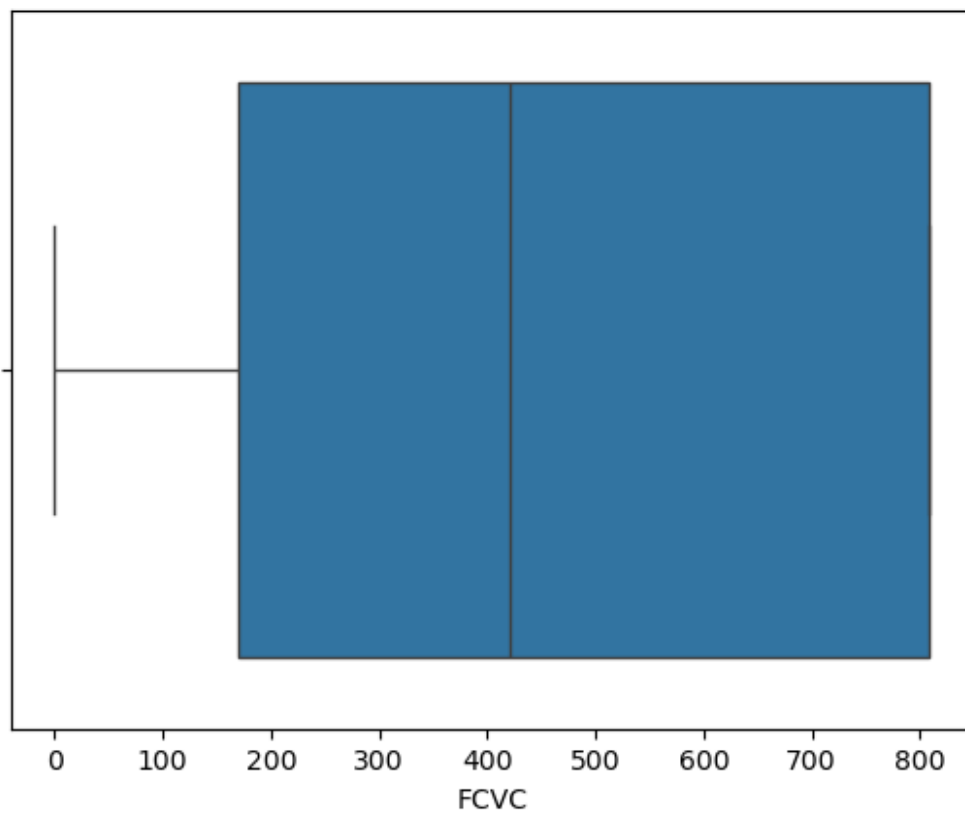


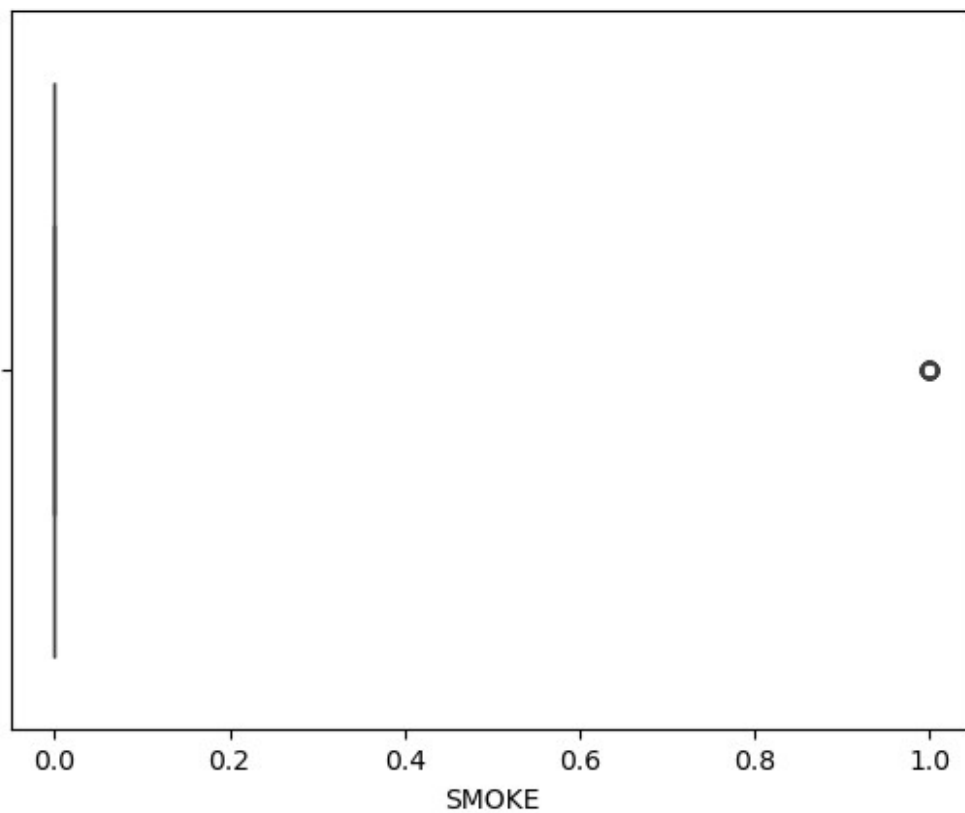
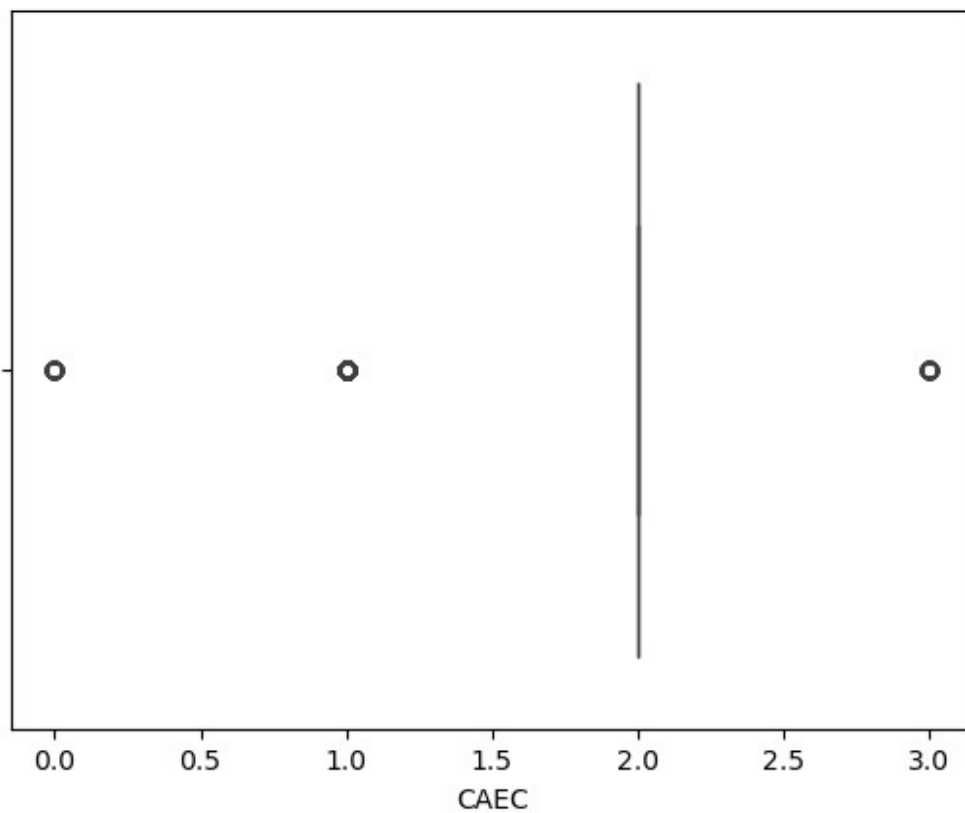
```
import warnings
warnings.filterwarnings("ignore")
for i in df.select_dtypes(include="int").columns:
    sns.boxplot(data=df, x=i)
    plt.show()
```

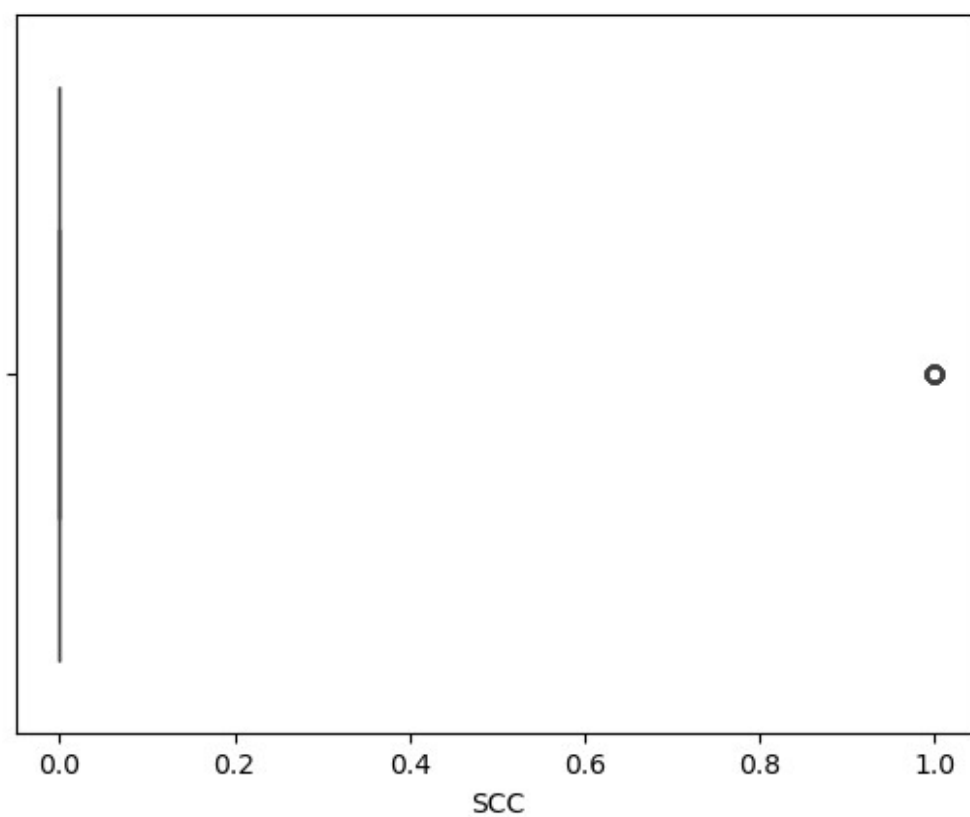
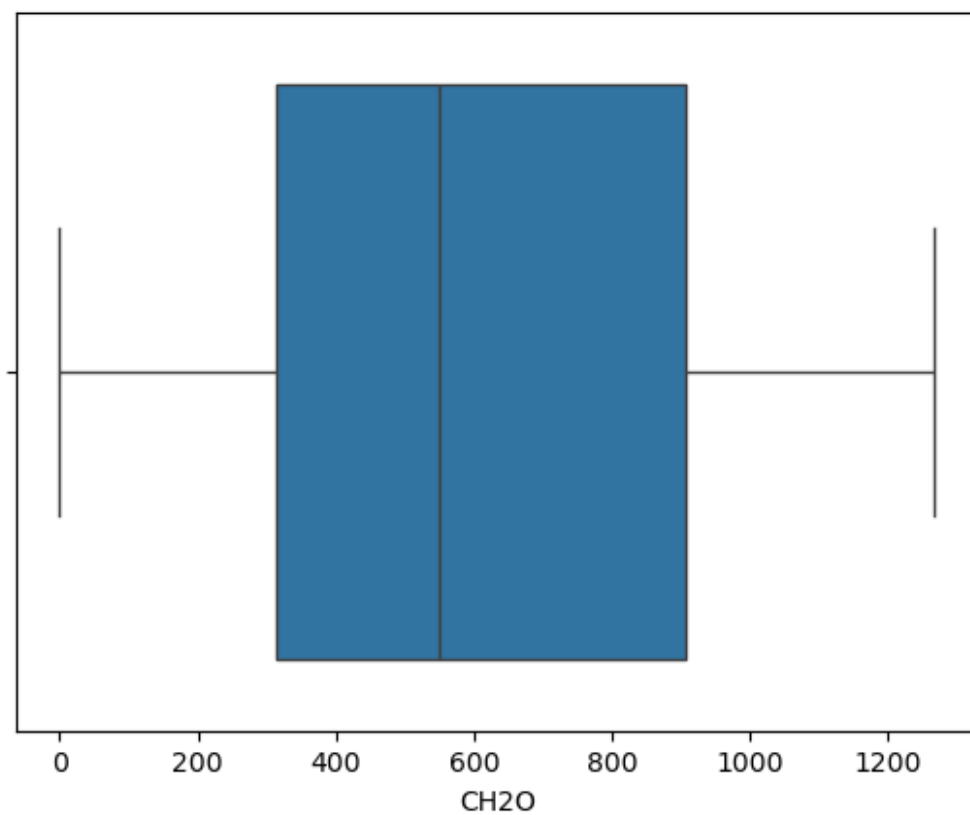


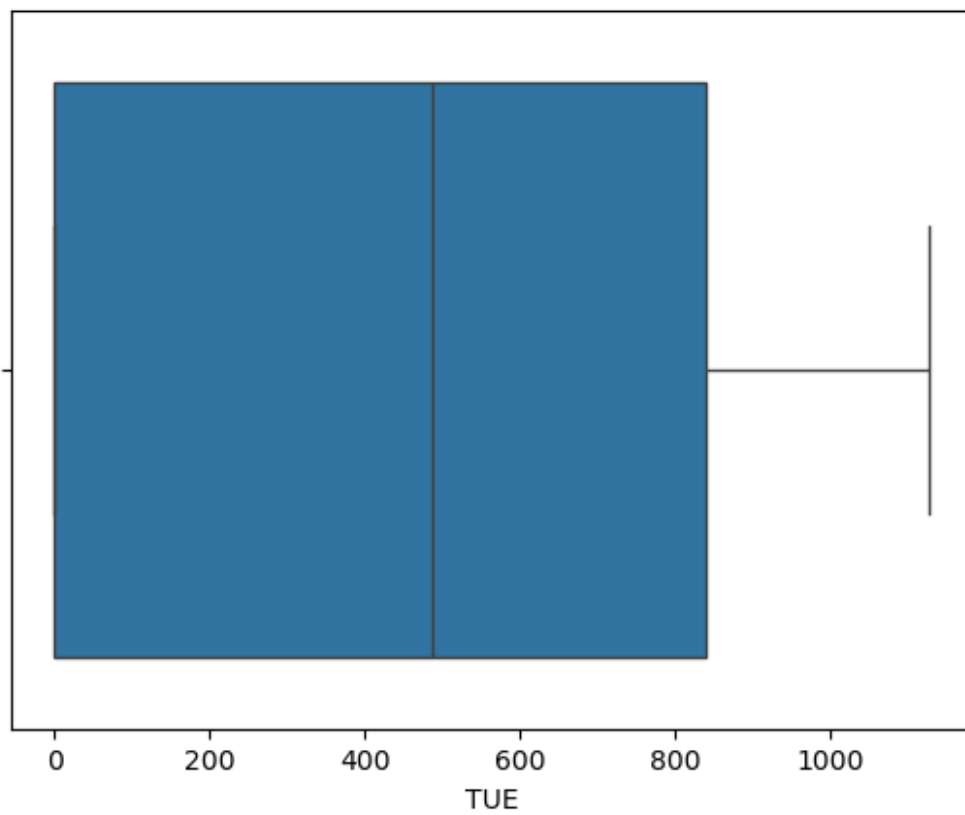
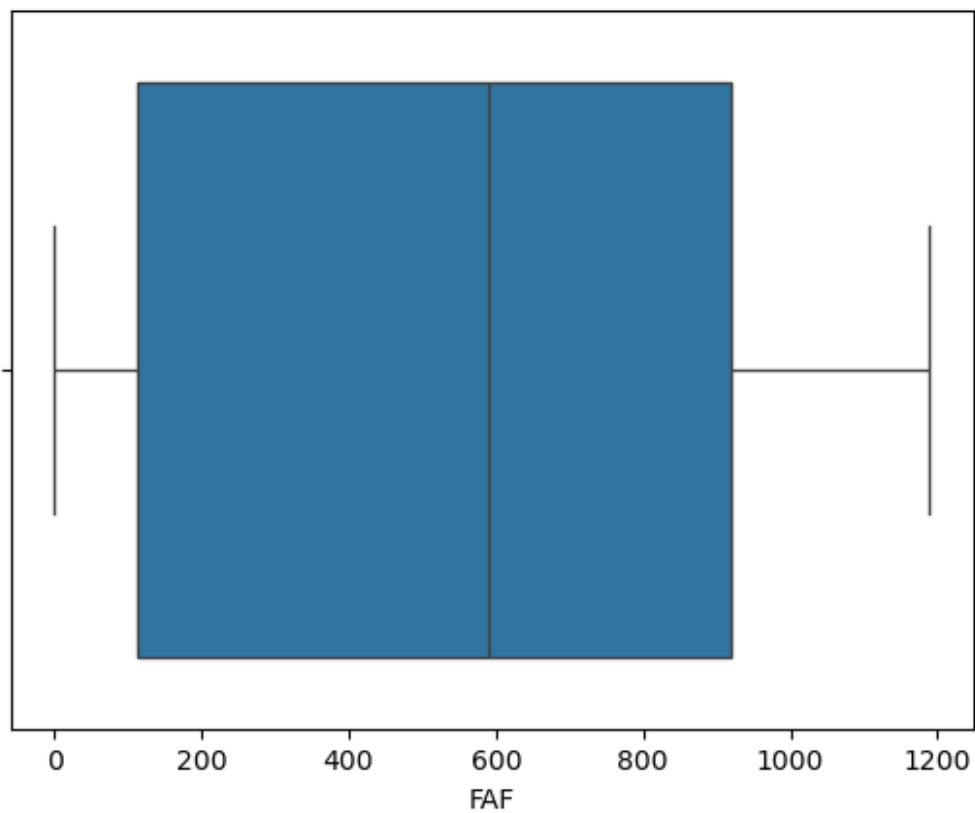


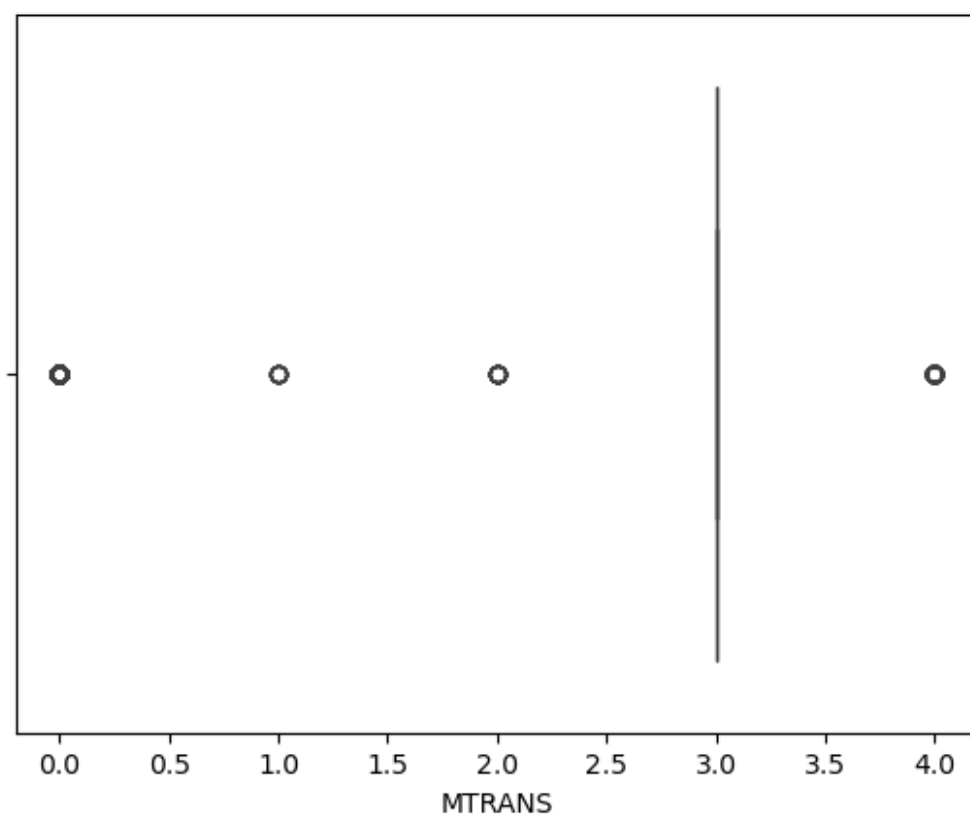
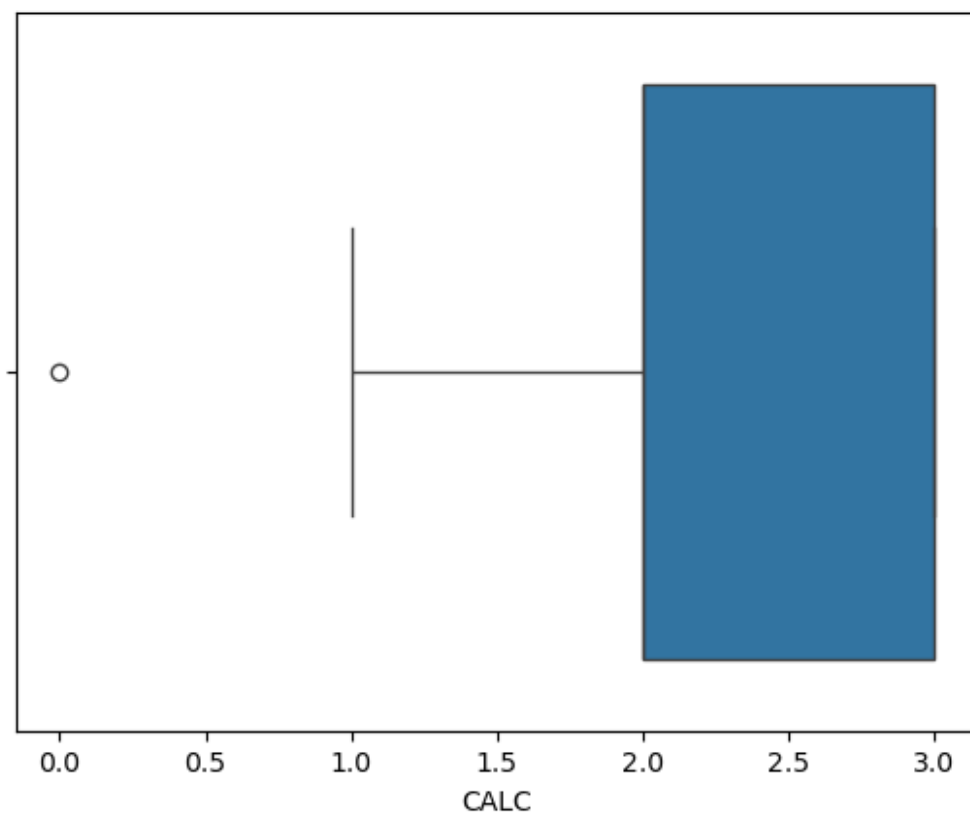


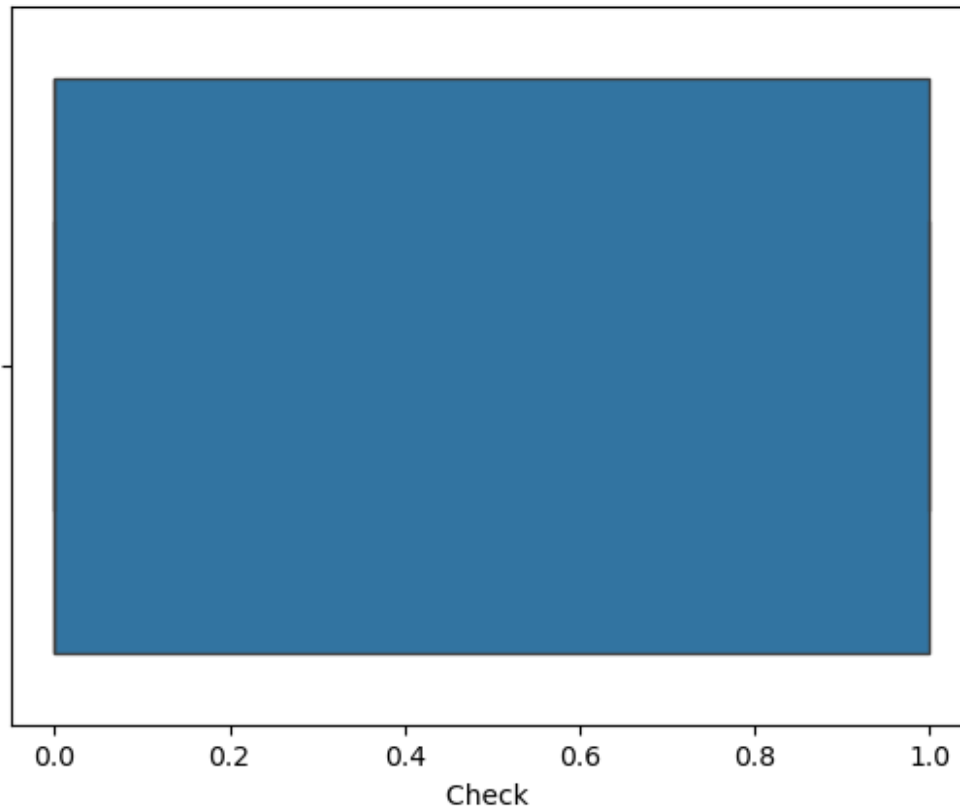












```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Example DataFrame (replace this with your actual DataFrame)
# df = pd.DataFrame({'SCC': [your_data_here]})

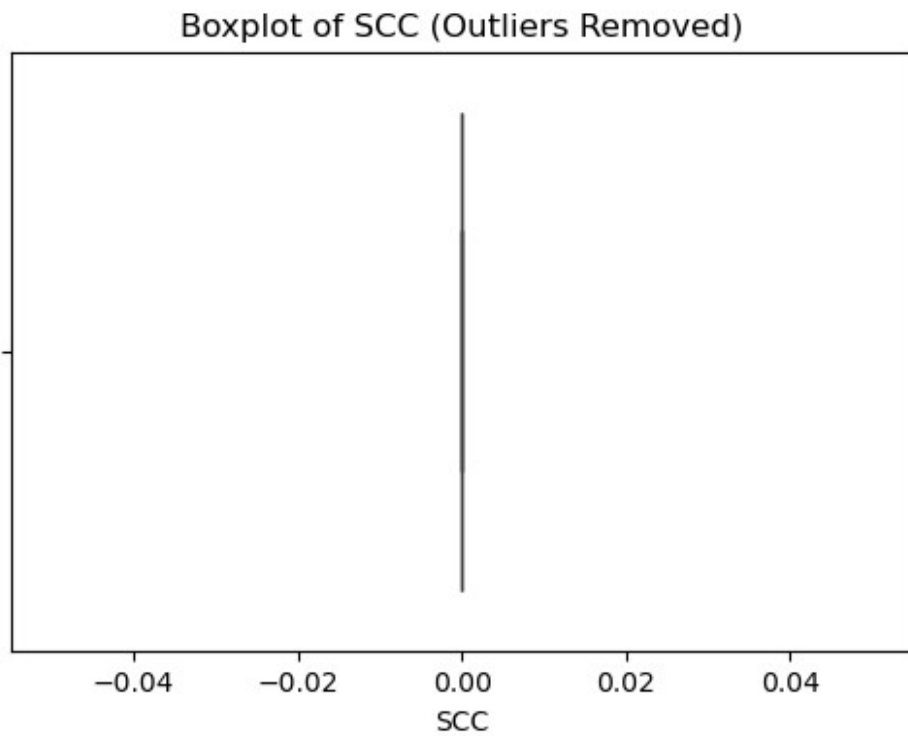
# Calculate the IQR
Q1 = df['SCC'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['SCC'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter the DataFrame to remove outliers
df_cleaned = df[(df['SCC'] >= lower_bound) & (df['SCC'] <=
upper_bound)]

# Boxplot for the cleaned 'SCC' column
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['SCC'])
plt.title('Boxplot of SCC (Outliers Removed)')
```

```
plt.xlabel('SCC')
plt.show()
```



```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Example DataFrame (replace this with your actual DataFrame)
# df = pd.DataFrame({'family_history': [your_data_here]})

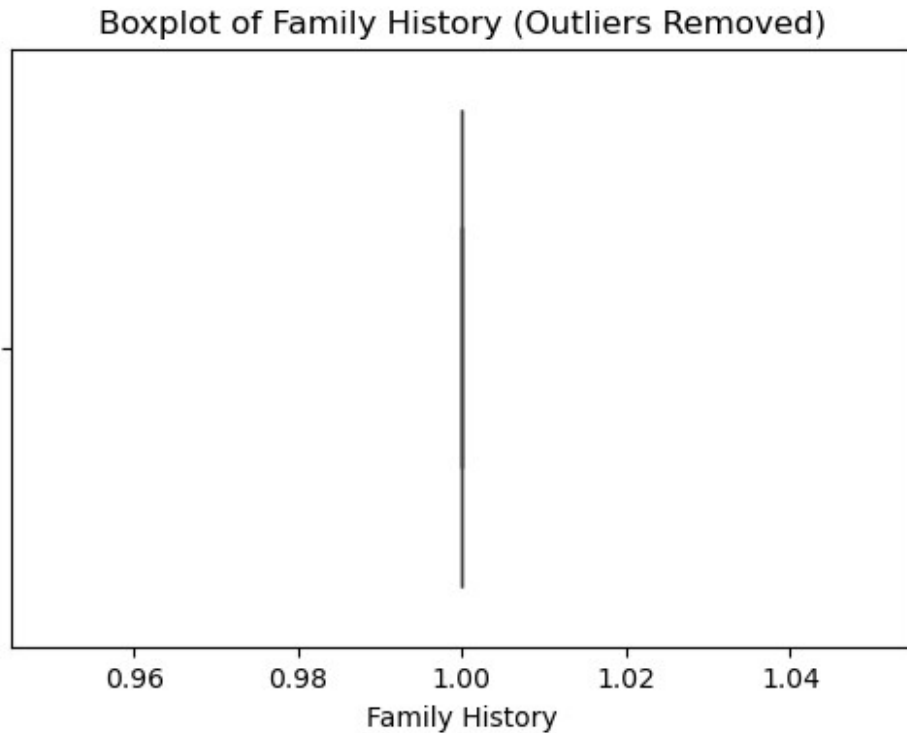
# Calculate the IQR
Q1 = df['family_history'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['family_history'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter the DataFrame to remove outliers
df_cleaned = df[(df['family_history'] >= lower_bound) &
                (df['family_history'] <= upper_bound)]

# Boxplot for the cleaned 'family_history' column
```

```
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['family_history'])
plt.title('Boxplot of Family History (Outliers Removed)')
plt.xlabel('Family History')
plt.show()
```



```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

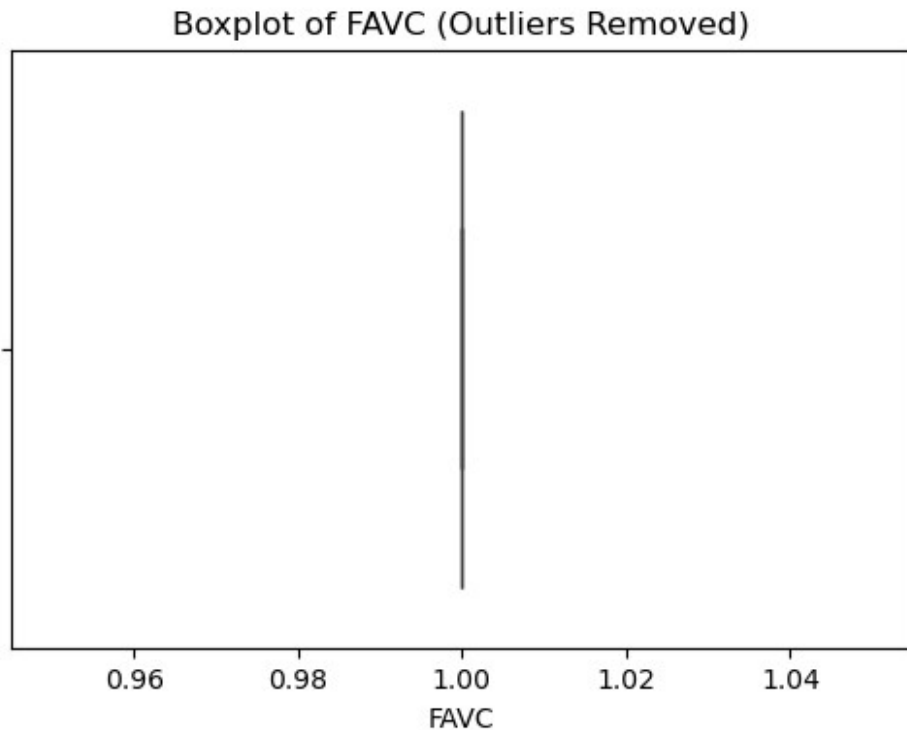
# Example DataFrame (replace this with your actual DataFrame)
# df = pd.DataFrame({'FAVC': [your_data_here]})

# Calculate the IQR
Q1 = df['FAVC'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['FAVC'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter the DataFrame to remove outliers
df_cleaned = df[(df['FAVC'] >= lower_bound) & (df['FAVC'] <=
upper_bound)]
```

```
# Boxplot for the cleaned 'FAVC' column
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['FAVC'])
plt.title('Boxplot of FAVC (Outliers Removed)')
plt.xlabel('FAVC')
plt.show()
```



```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure 'CAEC' is numeric and drop any non-numeric or NaN values
df['CAEC'] = pd.to_numeric(df['CAEC'], errors='coerce') # Convert to
numeric, invalid values become NaN
df.dropna(subset=['CAEC'], inplace=True) # Drop rows with NaN values
in 'CAEC' column

# Calculate the IQR for outlier detection
Q1 = df['CAEC'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['CAEC'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```

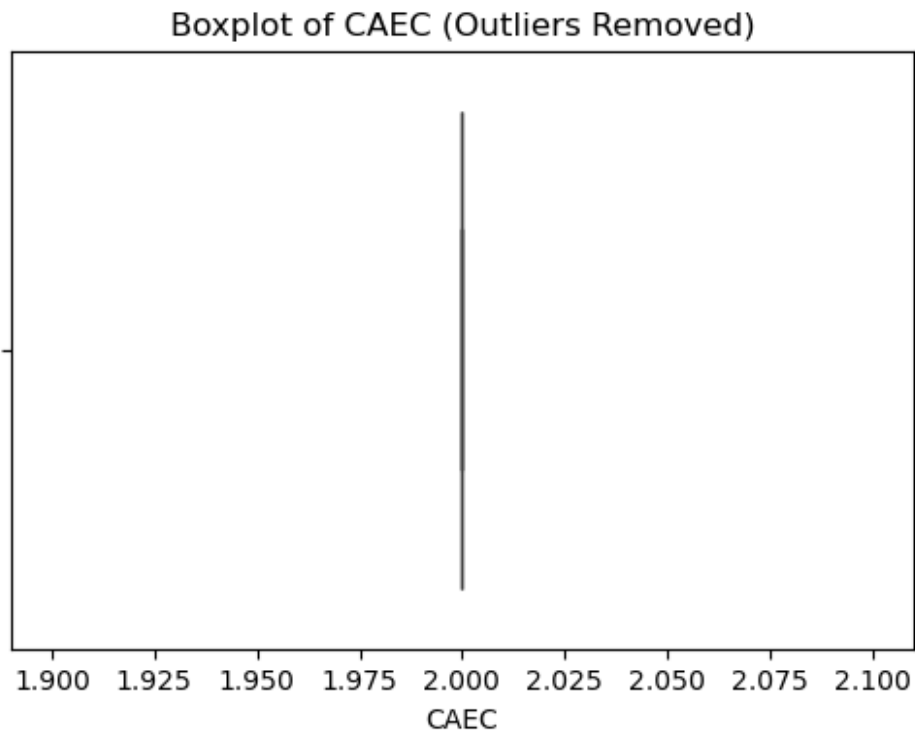
# Filter the DataFrame to remove outliers
df_cleaned = df[(df['CAEC'] >= lower_bound) & (df['CAEC'] <=
upper_bound)]

# Check how many rows were removed
print("Original data count:", len(df))
print("Cleaned data count:", len(df_cleaned))

# Boxplot for the cleaned 'CAEC' column (after removing outliers)
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['CAEC'])
plt.title('Boxplot of CAEC (Outliers Removed)')
plt.xlabel('CAEC')
plt.show()

Original data count: 2111
Cleaned data count: 1765

```



```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure 'SMOKE' is numeric and drop any non-numeric or NaN values
df['SMOKE'] = pd.to_numeric(df['SMOKE'], errors='coerce') # Convert
to numeric, invalid values become NaN
df.dropna(subset=['SMOKE'], inplace=True) # Drop rows with NaN values

```

```

in 'SMOKE' column

# Calculate the IQR for outlier detection
Q1 = df['SMOKE'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['SMOKE'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

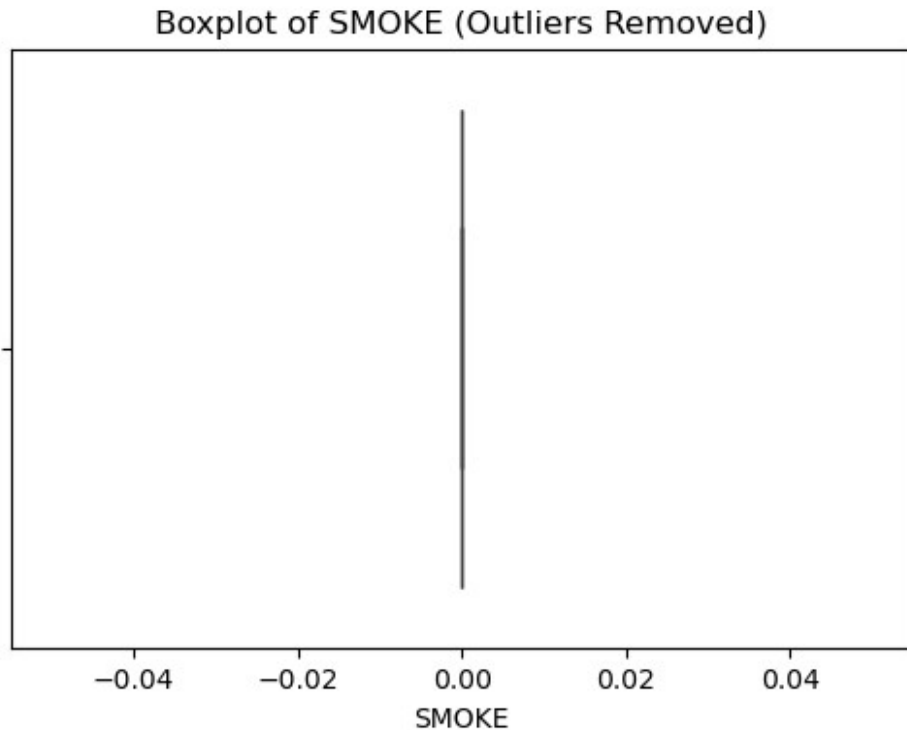
# Filter the DataFrame to remove outliers
df_cleaned = df[(df['SMOKE'] >= lower_bound) & (df['SMOKE'] <=
upper_bound)]

# Check how many rows were removed
print("Original data count:", len(df))
print("Cleaned data count:", len(df_cleaned))

# Boxplot for the cleaned 'SMOKE' column (after removing outliers)
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['SMOKE'])
plt.title('Boxplot of SMOKE (Outliers Removed)')
plt.xlabel('SMOKE')
plt.show()

Original data count: 2111
Cleaned data count: 2067

```



```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure 'SCC' is numeric and drop any non-numeric or NaN values
df['CALC'] = pd.to_numeric(df['CALC'], errors='coerce') # Convert to
numeric, invalid values become NaN
df.dropna(subset=['CALC'], inplace=True) # Drop rows with NaN values
in 'SCC' column

# Calculate the IQR for outlier detection
Q1 = df['CALC'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['CALC'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

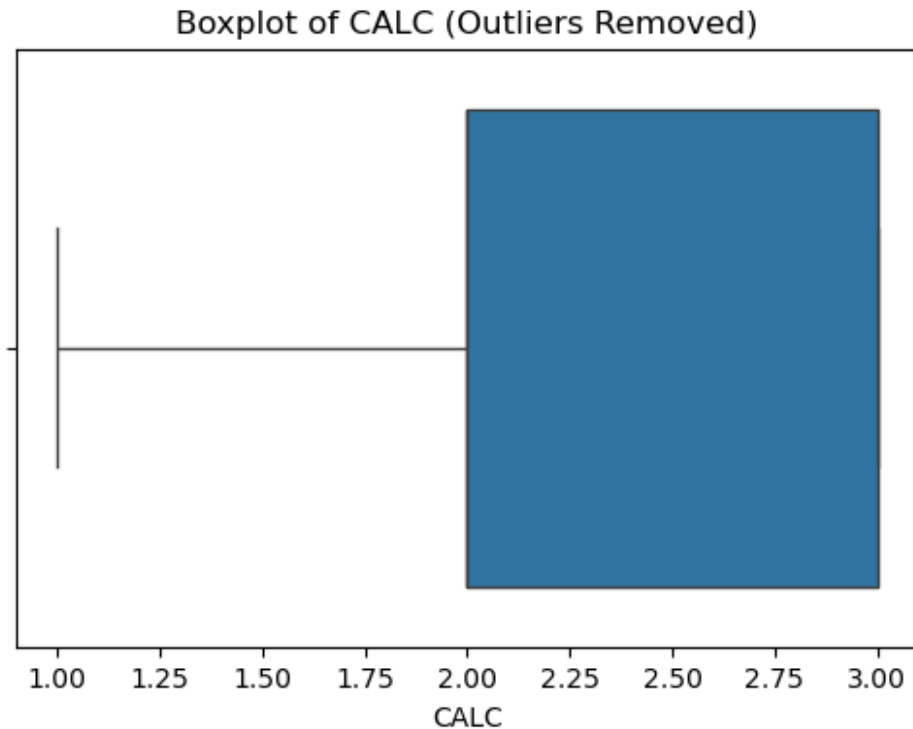
# Filter the DataFrame to remove outliers
df_cleaned = df[(df['CALC'] >= lower_bound) & (df['CALC'] <=
upper_bound)]

# Check how many rows were removed
print("Original data count:", len(df))
print("Cleaned data count:", len(df_cleaned))
```

```
# Boxplot for the cleaned 'SCC' column (after removing outliers)
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['CALC'])
plt.title('Boxplot of CALC (Outliers Removed)')
plt.xlabel('CALC')
plt.show()
```

Original data count: 2111

Cleaned data count: 2110



```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Ensure 'SCC' is numeric and drop any non-numeric or NaN values
df['MTRANS'] = pd.to_numeric(df['MTRANS'], errors='coerce') # Convert
to numeric, invalid values become NaN
df.dropna(subset=['MTRANS'], inplace=True) # Drop rows with NaN
values in 'SCC' column

# Calculate the IQR for outlier detection
Q1 = df['MTRANS'].quantile(0.25) # First quartile (25th percentile)
Q3 = df['MTRANS'].quantile(0.75) # Third quartile (75th percentile)
IQR = Q3 - Q1 # Interquartile range

# Define the lower and upper bounds for outliers
```

```

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

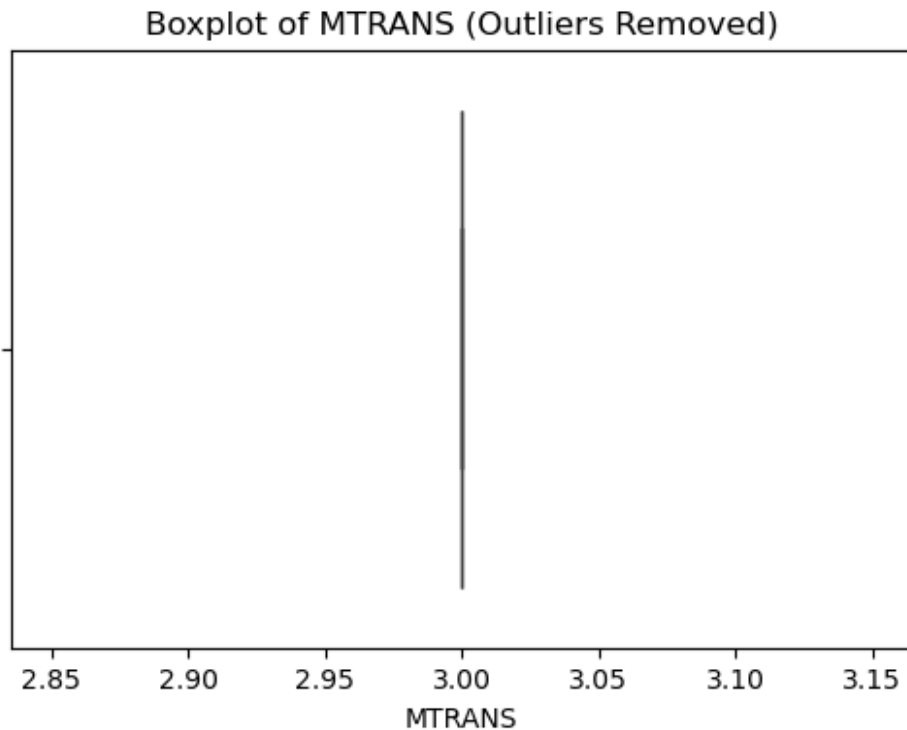
# Filter the DataFrame to remove outliers
df_cleaned = df[(df['MTRANS'] >= lower_bound) & (df['MTRANS'] <=
upper_bound)]

# Check how many rows were removed
print("Original data count:", len(df))
print("Cleaned data count:", len(df_cleaned))

# Boxplot for the cleaned 'SCC' column (after removing outliers)
plt.figure(figsize=(6, 4))
sns.boxplot(x=df_cleaned['MTRANS'])
plt.title('Boxplot of MTRANS (Outliers Removed)')
plt.xlabel('MTRANS')
plt.show()

Original data count: 2111
Cleaned data count: 1580

```



```

import pandas as pd
from sklearn.model_selection import train_test_split

# Load your dataset (replace with your file path)
data = pd.read_csv(r"F:\raji\obesity.csv")

```

```

# Define the features and target column
feature_columns = ([
    'Gender', 'Age', 'Height', 'Weight', 'family_history', 'FAVC',
    'FCVC', 'NCP', 'CAEC', 'SMOKE', 'CH20', 'SCC', 'FAF',
    'TUE', 'CALC', 'MTRANS'])

target_column = ['Obesity']

# Separate features (X) and target (y)
X = data[feature_columns]
y = data[target_column]

# Handle categorical variables by one-hot encoding
X_encoded = pd.get_dummies(X, drop_first=True)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42 # 20% test size
)

# Display shapes of the splits
print("Training features shape:", X_train.shape)
print("Testing features shape:", X_test.shape)
print("Training target shape:", y_train.shape)
print("Testing target shape:", y_test.shape)

Training features shape: (1688, 23)
Testing features shape: (423, 23)
Training target shape: (1688, 1)
Testing target shape: (423, 1)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Load your dataset (replace with your file path)
data = pd.read_csv(r"F:\raji\obesity.csv")

# Define the features and target column
feature_columns = ([
    'Gender', 'Age', 'Height', 'Weight', 'family_history', 'FAVC',
    'FCVC', 'NCP', 'CAEC', 'SMOKE', 'CH20', 'SCC', 'FAF',
    'TUE', 'CALC', 'MTRANS'])

target_column = ['Obesity']

# Separate features (X) and target (y)

```

```

X = data[feature_columns]
y = data[target_column]

# Handle categorical variables by one-hot encoding
X_encoded = pd.get_dummies(X, drop_first=True)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42 # 20% test size
)

# Initialize the DecisionTreeClassifier
dt_model = DecisionTreeClassifier(random_state=42)

# Train the model
dt_model.fit(X_train, y_train)

# Predict on the test data
y_pred = dt_model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print the accuracy
print(f"Accuracy of the Decision Tree model: {accuracy:.4f}")

```

Accuracy of the Decision Tree model: 0.9480

```

from sklearn.metrics import classification_report

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Classification Report:

	precision	recall	f1-score	support
Insufficient_Weight	0.93	0.98	0.96	56
Normal_Weight	0.92	0.89	0.90	62
Obesity_Type_I	0.96	0.96	0.96	78
Obesity_Type_II	0.96	0.95	0.96	58
Obesity_Type_III	1.00	1.00	1.00	63
Overweight_Level_I	0.89	0.91	0.90	56
Overweight_Level_II	0.96	0.94	0.95	50
accuracy			0.95	423
macro avg	0.95	0.95	0.95	423

weighted avg	0.95	0.95	0.95	423
--------------	------	------	------	-----

```
from sklearn.metrics import confusion_matrix

# Assuming y_test and y_pred are already defined
conf_matrix = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(conf_matrix)

Confusion Matrix:
[[55  1  0  0  0  0  0]
 [ 4 55  0  0  0  3  0]
 [ 0  0 75  2  0  0  1]
 [ 0  0  3 55  0  0  0]
 [ 0  0  0  0 63  0  0]
 [ 0  4  0  0  0 51  1]
 [ 0  0  0  0  0  3 47]]

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Example: Load a dataset (you can replace this with your own dataset)
# For demonstration, let's use the Iris dataset
from sklearn.datasets import load_iris
data = load_iris()

# Convert the dataset to a DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# Define features (X) and target (y)
X = df.drop('target', axis=1) # Features
y = df['target'] # Target labels

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train, y_train)

# Make predictions
y_pred = svm_model.predict(X_test)
```

```
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Generate and display the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
```

Model Accuracy: 100.00%

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load your dataset (replace with your file path)
data = pd.read_csv(r"F:\raji\obesity.csv")

# Define the features and target column
feature_columns = (
    'Gender', 'Age', 'Height', 'Weight', 'family_history', 'FAVC',
    'FCVC', 'NCP', 'CAEC', 'SMOKE', 'CH20', 'SCC', 'FAF',
    'TUE', 'CALC', 'MTRANS'])

target_column = ['Obesity']

# Separate features (X) and target (y)
```

```

X = data[feature_columns]
y = data[target_column]

# Handle categorical variables by one-hot encoding
X_encoded = pd.get_dummies(X, drop_first=True)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42 # 20% test size
)

# Initialize the RandomForestClassifier
rf_model = RandomForestClassifier(random_state=42)

# Train the model
rf_model.fit(X_train, y_train)

# Predict on the test data
y_pred = rf_model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print the accuracy
print(f"Accuracy of the Random Forest model: {accuracy:.4f}")

Accuracy of the Random Forest model: 0.9551

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Classification Report:

	precision	recall	f1-score	support
Insufficient_Weight	0.98	0.96	0.97	56
Normal_Weight	0.87	0.95	0.91	62
Obesity_Type_I	0.99	0.94	0.96	78
Obesity_Type_II	0.97	0.98	0.97	58
Obesity_Type_III	1.00	1.00	1.00	63
Overweight_Level_I	0.96	0.88	0.92	56
Overweight_Level_II	0.92	0.98	0.95	50
accuracy			0.96	423
macro avg	0.96	0.96	0.95	423
weighted avg	0.96	0.96	0.96	423

```

from sklearn.metrics import confusion_matrix

# Assuming y_test and y_pred are already defined

```

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:")
```

```
print(conf_matrix)
```

Confusion Matrix:

```
[[54  2  0  0  0  0  0]
 [ 1 59  0  0  0  1  1]
 [ 0  1 73  2  0  0  2]
 [ 0  0  1 57  0  0  0]
 [ 0  0  0  0 63  0  0]
 [ 0  6  0  0  0 49  1]
 [ 0  0  0  0  0  1 49]]
```